



Digital IC Design & Verification
Physical Design
Lab Manual # 03 | Back-End

Release: 1.0

Date: 14-May-2026

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is intended for recipients only. It is not allowed to copy, modify, distribute, or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
0.1	17/01/2024	Hira Sohail	Designing Complete Manual	Dr. Waqar
0.2	05/05/2025	Hira Sohail	Revision in Manual	Sharjeel Khilji
0.3	03/10/2025	Fawad Ahmed	Revision in Manual	Sharjeel Khilji
1.0	14/05/2026	Fawad Ahmed	- Transform into Script-Based Back-End Design - Added Tempus Tool for final STA	Sharjeel Khilji
1.1	29/06/2026	Fawad Ahmed	Added the Conformal Flow	Sharjeel Khilji
1.2	29/07/2026	Fawad Ahmed	Added the LVS Flow	Sharjeel Khilji



Table of Contents

Objective	3
Tools	4
Instructions	4
1. Introduction	4
2. Learning Objectives	5
3. The Flow of Backend Design	6
4. Design Inputs	7
4.1. Netlist (.v)	7
4.2. Synopsys Design Constraint (.sdc)	7
4.3. Library Exchange Format (.lef)	7
4.4. Library Timing Files (.lib)	7
4.5. Multi-Mode Multi-Corner (MMMC) Setup File (.view)	8
4.6. Pin Assignment / I/O File (.io)	8
4.7. Standard Cell SPICE Library (.spi / .cdl)	8
4.8. Physical Verification Rule File (.pvl)	8
5. Executing the Backend Flow	8
5.1. Design Initialization & Data Import	9
5.2. Tool Settings, Timing Setup, and Initialization	10
5.2.1. Step 1: Base Units and Timing Engine Configuration	10
5.2.2. Step 2: Loading Inputs and Database Compilation (init_design)	11
5.2.3. Step 3: Global Net Connectivity	12
5.3. Floorplanning & Pin Placement	13
5.3.1. Step 1: Defining the Silicon Boundary	13
5.3.2. Step 2: Placing the I/O Pins	15
5.4. Power-Planning	16
5.4.1. Step 1: Setting Layers & Core Rings	16
5.4.2. Step 2: Follow Pins (Standard Cell Power Rails)	18
5.4.3. Step 3: End Caps (Boundary Protection)	19
5.4.4. Step 4: Vertical Power Stripes	19
5.5. Placement & Pre-CTS Optimization	21
5.5.1. Step 1: Congestion-Aware Placement & Optimization	21
5.5.2. Step 2: Adding Physical Tie Cells	23
5.5.3. Step 3: Pre-CTS Design Rule Violation (DRV) Fixing	24
5.6. Clock Tree Synthesis (CTS) & Post CTS Optimization	24
5.6.1. Step 1: Defining the Clock Tree Specification	24
5.6.2. Step 2: Building and Balancing the Tree (ccopt_design)	25



5.6.3. Step 3: Post-CTS Optimization	26
5.7. Signal Routing and Post-Route Optimization.....	27
5.7.1. Step 1: Configuring Physics Awareness and ECO Settings.....	27
5.7.2. Step 2: Global and Detailed Routing (route_opt_design).....	27
5.7.3. Step 3: Post-Route Timing Optimization	28
5.8. Design for Manufacturing (DFM)	29
5.8.1. Step 1: Disabling Routing Engines and Configuring DFM Variables	29
5.8.2. Step 2: Multi-Cut Via Insertion and Wire Spreading	29
5.8.3. Step 3: Restoring the Database State	30
5.9. Physical Continuity (Filler and Decap Insertion)	30
5.9.1. The Physics of Continuity (Why do we need fillers?)	30
5.9.2. Inserting Decaps and Fillers.....	31
5.9.3. Standard Fillers vs. Decaps Cells	32
5.10. Physical Verification (DRC/Connectivity) and Power Analysis.....	32
5.10.1. Design Rule Check (DRC)	32
5.10.2. Connectivity Check.....	32
5.10.3. Power Analysis	33
5.11. Backend Formal Verification (LEC)	34
5.12. Signoff Final STA	34
5.13. Final Design Export (Handoff Deliverables)	35
5.13.1. Step 1: Exporting the Netlist and Parasitics.....	35
5.13.2. Step 2: Generating the GDSII (The "Tapeout" File).....	35
5.14. Physical Verification (LVS Signoff)	36
5.14.1. The LVS Control File (.ctl)	36
5.14.2. The Physical Verification Rule File (.pvl)	37
5.14.3. Executing the LVS Check.....	38
5.14.4. Analyzing the LVS Results and Reports	38
5.14.4.1. The Main Report File (lvs.rep).....	39
5.14.4.2. The Detailed Error Classification (lvs.rep.cls).....	39
5.14.4.3. The Shorts Report (lvs.rep.shortcuts)	39
5.14.4.4. The Electrical Rule Check Summary (erc.sum)	39
5.14.4.5. The Soft Check Report (counter.softchk)	40
5.15. Signoff STA (Tempus)	40
5.15.1. Invoking Tempus.....	40
5.15.2. Engine Configuration & Data Loading	40
5.15.3. Loading Parasitics (SPEF)	41
5.15.4. Generating Signoff Scorecards	41



6. Lab Tasks	42
7. Resources	44



Objective

The objective of this lab is to

- **Implement a complete Physical Design flow** in Cadence Innovus, transforming a gate-level netlist into a fully routed, GDSII-ready layout.
- **Master Floorplanning and Power Planning** techniques, including the creation of core rings, stripes, and explicit I/O pin placement for a 4-bit counter
- **Execute Clock Tree Synthesis (CTS)** to balance clock distribution and minimize skew across the register-to-register paths.
- **Perform Design-for-Manufacturing (DFM) Optimizations**, such as wire spreading and multi-cut via insertion, to improve silicon yield and reliability.
- **Perform Formal Verification (LEC)** using Cadence Conformal to mathematically prove that physical optimizations at intermediate checkpoints (Placement, CTS, Routing) do not alter the Golden synthesis logic.
- **Execute Signoff Physical Verification (LVS)** using Cadence PVS to ensure the final exported geometric layout (GDSII) perfectly matches the logical connectivity of the routed netlist.
- **Conduct Signoff Static Timing Analysis (STA)** using Cadence Tempus to verify timing closure across all MMMC views using final parasitics (SPEF) and routed netlists

Tools

- Cadence Innovus
- Cadence Conformal
- Cadence Pegasus
- Cadence Tempus

Instructions

The students are required to design and submit the following files for evaluation of this lab which is given below:

- The report should be in detail. Each step and all the commands should be explained properly. Also, the results and its effects should be explained.
- Your files should be submitted in the format, i.e.
 - Name – Module – Lab Number.zip [Includes all required sv, rpt, sdc, tcl files]
 - Name – Module – Lab Number.pdf [Your Report]



1. Introduction

Physical Design is the comprehensive process of transforming a synthesized, gate-level netlist into a geometric representation that can be manufactured in a silicon foundry. While synthesis focuses on logical correctness and timing budgets, Physical Design (often called "Back-End" design) addresses the physical reality of the chip, including the placement of millions of transistors and the routing of microscopic copper wires across multiple metal layers.

Using industry-leading tools like **Cadence Innovus**, the design undergoes a rigorous physical transformation through several critical stages:

- **Floorplanning & Power Planning:** Defining the chip boundary and constructing a robust Power Distribution Network (PDN) to ensure every logic gate receives stable voltage.
- **Placement:** Determining the optimal X/Y coordinates for every standard cell to minimize congestion and wire length.
- **Clock Tree Synthesis (CTS):** Engineering a balanced distribution network for the clock signal to ensure it reaches every flip-flop simultaneously, minimizing skew.
- **Routing:** Automatically drawing the actual metal interconnections between gates while adhering to strict Design Rule Manual (DRM) spacing requirements.

The process concludes with rigorous **Signoff Audits**. This includes **Formal Verification** using **Cadence Conformal (LEC)** to mathematically prove that the physical transformations did not alter the original Boolean logic, **Physical Verification (LVS)** using **Cadence PVS** to ensure the geometric layout perfectly matches the intended schematic connectivity, alongside **Signoff Static Timing Analysis (STA)** in **Cadence Tempus** to guarantee performance across all environmental extremes. Once all logical and timing signoff criteria are flawlessly met, the design is officially "Taped Out" for manufacturing.

2. Learning Objectives

After completing this lab on **Advanced ASIC Back-End Implementation and Signoff**, you will be able to:

- **Transform** a gate-level netlist into a physical layout by executing the full **Place and Route (P&R)** flow within the Cadence Innovus environment.
- **Construct** a robust **Power Distribution Network (PDN)**, including core rings, power stripes, and follow-pins, to ensure stable voltage delivery and minimize IR drops across the chip.
- **Synthesize** a balanced **Clock Tree (CTS)** using the CCOpt engine to minimize clock skew and latency while managing the physical power consumption of the clock buffers.
- **Mitigate** routing congestion and design rule violations (DRVs) by optimizing cell placement and utilizing multi-layer metal interconnects.
- **Extract** parasitic RC values (SPEF) from the final routed design to perform high-accuracy **Signoff Static Timing Analysis (STA)** in Cadence Tempus.



- **Validate** timing closure across **Multi-Mode Multi-Corner (MMMC)** views, ensuring the design meets all setup, hold, and design rule requirements under worst-case and best-case silicon conditions.
- **Perform** Formal Verification (LEC) at critical physical checkpoints to mathematically prove that aggressive backend optimizations (buffer insertion, gate resizing, and clock tree routing) did not alter the original Golden synthesis logic.
- **Configure and execute Layout Versus Schematic (LVS)** physical verification using **Cadence PVS** to guarantee that the final **geometric layout (GDSII)** perfectly matches the intended logical netlist.
- **Evaluate** the "Final-Inch" trade-offs between wire length, congestion, and timing slack to determine if a design is ready for GDSII tape-out.

3. The Flow of Backend Design

The Front-End of ASIC design (RTL coding and Logic Synthesis) answers the question: "What does the chip do?" The Back-End, or Physical Design, answers the question: "How do we physically build it on silicon?"

Physical design is a highly complex optimization problem. To solve it, EDA tools like Cadence Innovus break the process down into a strict sequence of steps known as the **Physical Design Flow**. Each step builds upon the previous one, transitioning the design from a logical netlist into a printable geometric layout.

The standard industry flow consists of the following seven major phases:

- **Design Import (Initialization):** The synthesized logical netlist (.v), timing constraints (.sdc), and physical technology libraries (.lef, .lib) are loaded into the implementation tool to create the initial database.
- **Floorplanning:** The physical dimensions of the chip (the core and die area) are established. Input/Output (I/O) pins are placed along the boundaries, and large macro blocks (if any, such as SRAM) are positioned. This dictates the available area for standard cell placement.
- **Power Planning (Power Distribution Network - PDN):** Before any logic is placed, a robust metallic grid must be constructed to deliver VDD (Power) and VSS (Ground) to every region of the chip. This involves drawing thick global rings, vertical stripes, and horizontal standard-cell rails to prevent voltage drops (IR Drop) and electromigration.
- **Placement:** The tool automatically assigns a specific physical coordinate to every logic gate (standard cell) in the netlist. The placement engine attempts to cluster connected cells close together to minimize wire length, congestion, and timing delays.
- **Clock Tree Synthesis (CTS):** The clock signal must arrive at every sequential element (flip-flop) at exactly the same time. The CTS engine physically routes the clock net and inserts a balanced "tree" of buffers to ensure minimal clock skew across the entire chip.



- **Routing:** The NanoRoute engine connects all remaining signal pins (data nets) using the available metal layers (e.g., M1 through M9). The router must navigate around the power grid and follow strict physical design rules (DRC) to prevent short circuits and mitigate signal interference (crosstalk).
- **Formal Verification (LEC Checkpoints):** Exporting intermediate gate-level netlists after Placement, CTS, and Routing to mathematically prove that the physical transformations did not break the Boolean logic equations of the chip.
- **Signoff and Analysis:** The fully routed design undergoes final, rigorous verification. This includes parasitic RC extraction, Static Timing Analysis (STA) with Tempus, and dynamic power analysis with Voltus and Physical Verification (LVS) using PVS/Pegasus to confirm the drawn geometric shapes perfectly match the logical netlist. Once verified, the geometric data is exported as a **GDSII** file, which is sent to the semiconductor foundry for manufacturing.

4. Design Inputs

Before launching the Cadence Innovus implementation tool, the physical design engineer must gather a specific set of input files. Innovus requires three distinct types of information to build a chip: the **Logical Intent** (what the chip does), the **Physical Rules** (the sizes and shapes of the manufacturing process), and the **Timing Constraints** (how fast it must run).

For this design utilizing the TSMC 65nm process, the following files serve as the foundational inputs:

4.1. Netlist (.v)

The gate-level structural Verilog file generated during the Front-End Logic Synthesis phase (e.g., using Cadence Genus).

4.2. Synopsys Design Constraint (.sdc)

The text file that dictates the electrical "speed limits" and performance requirements of the design. It defines the target clock frequency, input/output delays, and critical Design Rule Violations (DRVs) such as maximum transition and capacitance.

4.3. Library Exchange Format (.lef)

The physical blueprints provided by the semiconductor foundry (TSMC). It defines the physical dimensions of the standard cells, pin locations, and the design rules for the metal routing layers.

4.4. Library Timing Files (.lib)

The electrical characterization database for standard cells. It tells the tool how fast it is and how much power it burns. Because transistors behave differently under different environmental conditions, multiple .lib files are loaded to represent various PVT



Corners (Process, Voltage, and Temperature), such as typical, worst (slow), and best (fast).

4.5. Multi-Mode Multi-Corner (MMMC) Setup File (.view)

A master configuration script that ties all the constraints and timing libraries together. Modern chips operate in different "Modes" (e.g., Functional Mode, Test Mode) and across different "Corners" (e.g., Worst-case, Best-case).

4.6. Pin Assignment / I/O File (.io)

A text-based configuration file that defines the exact physical locations, ordering, and metal layers for the chip's input and output ports. It forces Innovus to place specific pins on specific sides of the floorplan boundary.

4.7. Standard Cell SPICE Library (.spi / .cdl)

The transistor-level netlist of the standard cells. While Innovus uses .lef and .lib for placement and timing, the physical verification tool (Cadence PVS) requires this SPICE file during Layout Versus Schematic (LVS) to map the Verilog gate names to their actual internal P-MOS and N-MOS transistor structures.

4.8. Physical Verification Rule File (.pvl)

The foundry-provided rule deck for Cadence PVS. It contains the strict mathematical algorithms the tool uses to extract electrical connectivity and devices directly from the drawn geometric shapes (GDSII) during LVS signoff.

5. Executing the Backend Flow

In this section, we will execute the complete physical design flow for the 4-bit counter using the Cadence Innovus command-line interface. While physical design can be executed via the graphical user interface (GUI), utilizing **Tcl (Tool Command Language) scripts** is the industry standard. Scripting ensures that the flow is repeatable, automated, and less prone to human error. We will build the chip sequentially, verifying the integrity of the design at the end of each major phase. For this, we should invoke Cadence Innovus by using the command below:

```
*****
*****      Welcome to Cadence Tools      *****
*****
[cc@ncdc-0081 workspace]$ innovus -stylus
```

Figure 1: Invoking Innovus

After initializing the Innovus, you observe the output below in terminal as well as GUI.



```
Cadence Innovus(TM) Implementation System.
Copyright 2021 Cadence Design Systems, Inc. All rights reserved worldwide.

Version:      v21.18-s099_1, built Tue Jul 18 13:03:50 PDT 2023
Options:      -stylus
Date:         Wed May 13 15:33:54 2026
Host:         ncdc-0081 (x86_64 w/Linux 4.18.0-372.32.1.el8_6.x86_64) (8cores*16cpus*12th Gen Intel(R) Core(TM) i7-12700 25600KB)
OS:           Red Hat Enterprise Linux 8.6 (Ootpa)

License:

[15:33:54.301481] Configured Lic search path (21.01-s002): 5280@10.19.32.90

invs  Innovus Implementation System 21.1  checkout succeeded
8 CPU jobs allowed with the current license(s). Use set_multi_cpu_usage to set your required CPU count.

Create and set the environment variable TMPDIR to /tmp/innovus_temp_144445_ncdc-0081_cc_4GEWp9.

Change the soft stacksize limit to 0.2%RAM (127 mbytes). Set global soft_stack_size_limit to change the value.
[INFO] Loading Pegasus 23.13 fill procedures

**INFO:  MMMC transition support version v31-84

@innovus 1> █
```

Figure 2: Innovus in Terminal

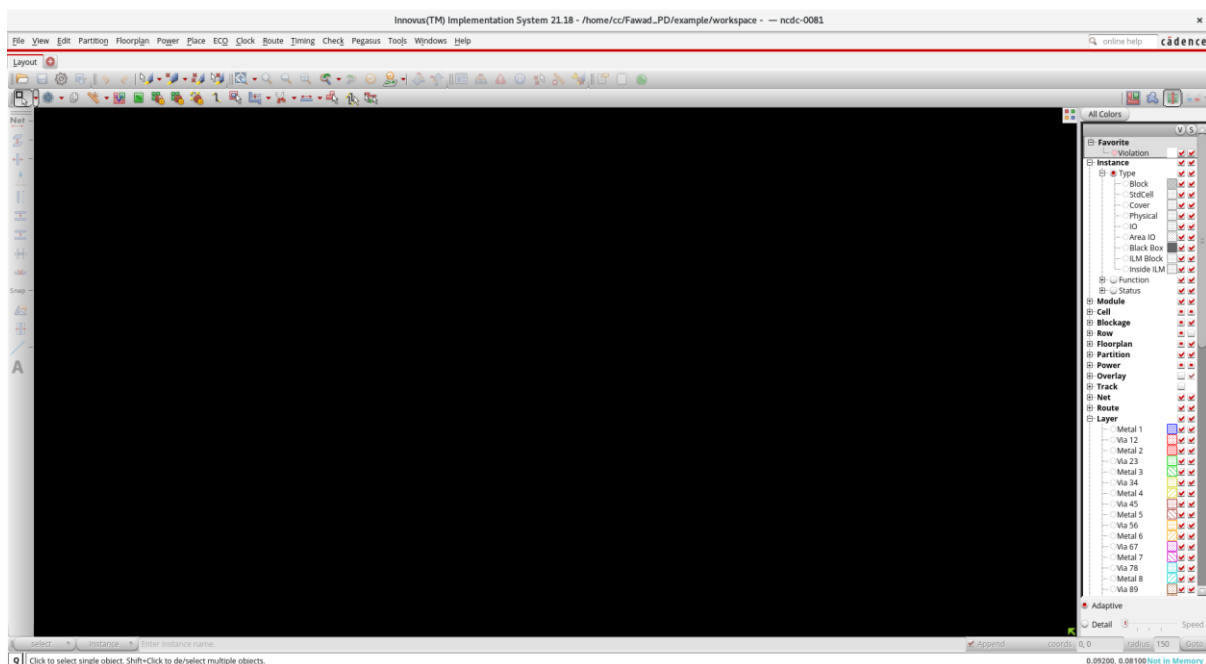


Figure 3: Innovus GUI

In the terminal, you will execute the following commands one by one & observe each output.

5.1. Design Initialization & Data Import

Before physical placement can begin, the tool must be initialized. This involves setting up directory paths, defining the TSMC 65nm technology variables, and loading the foundational design inputs (Netlist, LEF, MMMC setup, and I/O assignments). Execute the following script block to establish the environment variables and instruct Innovus to read the front-end design files.



```
# =====
# 1. PROJECT VARIABLES & SETUP
# =====

# -----
# ⚠ IMPORTANT: UPDATE THIS DIRECTORY PATH FOR YOUR SPECIFIC PC ⚠
# -----
# set WORK_DIR: Defines the root workspace directory. All other paths derive from this.
set WORK_DIR "/home/cc/Fawad_PD/example/"
# -----

# set design(TOPLEVEL): Defines the name of the top module to be used in output files.
set design(TOPLEVEL) "counter"

# --- Output Infrastructure & Directory Creation ---
# Define paths for saving Innovus outputs (GDS, Netlists) and reports.
set INNOVUS_DATA_DIR "${WORK_DIR}/export/Innovus_Data"
set design(export_dir) "${INNOVUS_DATA_DIR}/outputs"
set design(reports_dir) "${INNOVUS_DATA_DIR}/reports"

# file mkdir: Instructs the OS to create the folder hierarchy to avoid 'directory not found' errors.
file mkdir $INNOVUS_DATA_DIR \
    ${design(export_dir)}/pnr \
    ${design(reports_dir)}/verify \
    ${design(reports_dir)}/signoff

# --- Inputs ---
# Link the gate-level netlist generated by the Genus Synthesis tool.
set design(postsyn_netlist) "${WORK_DIR}/export/Synthesis_Data/netlists/${design(TOPLEVEL)}_physical_synth.v"
# Link the MMMC (Multi-Mode Multi-Corner) file for timing constraints (SDC) and RC corners.
set design(mmmc_view_file) "${WORK_DIR}/inputs/mmmc_master.view"
# Link the IO file which dictates exact coordinates for input/output pins.
set design(io_file) "${WORK_DIR}/inputs/${design(TOPLEVEL)}.io"
# Link the LEF file containing the physical dimensions, shapes, and layer rules of standard cells.
set tech(ALL_LEFS) "${WORK_DIR}/libraries/lef/tcbn65lp_9lmT2.lef"

# --- Cell Library Definitions ---
# Map the logical power/ground pins to physical cell references.
set tech(STANDARD_CELL_VDD) "VDD"
set tech(STANDARD_CELL_GND) "VSS"
# Specify the cells used at the ends of standard cell rows to prevent well-spacing DRCs.
set tech(ENDCAPS_right) "DCAP"
set tech(ENDCAPS_left) "DCAP"
# Specify cells used to tie logic '1' and logic '0' without connecting directly to PG rails (prevents ESD damage).
set tech(TIEHI) "TIEH"
set tech(TIELO) "TIEL"
# Limit the fanout of tie cells so they don't get overloaded.
set tech(TIE_MAX_FANOUT) 10
# List of Decoupling Capacitor and Filler cells to fill empty spaces in the floorplan.
set tech(DECAP) "DCAP64 DCAP32 DCAP16 DCAP8 DCAP4 DCAP"
set tech(FILLERS) "FILL64 FILL32 FILL16 FILL8 FILL4 FILL2 FILL1"

# --- Global Net Definitions ---
# Explicitly name the global power and ground nets for routing.
set design(digital_gnd) "VSS"
set design(digital_vdd) "VDD"
set design(all_ground_nets) $design(digital_gnd)
set design(all_power_nets) $design(digital_vdd)
set design(core_ring_nets) "$design(digital_gnd) $design(digital_vdd)"
```

Figure 4: Setting the Variables

You should know these basic commands as you have learnt in Scripting Module.

5.2. Tool Settings, Timing Setup, and Initialization

With the project variables defined, we must configure the internal engines of Innovus before we construct the physical database. This involves setting physical units, configuring the timing analysis engine, loading the input files, and officially initializing the design in memory.



5.2.1. Step 1: Base Units and Timing Engine Configuration

Before Innovus can calculate delay or power, it needs to know the mathematical units to use. Furthermore, we must tell the Static Timing Analysis (STA) engine how to handle physical manufacturing variations.

```
# Establish base units for the database to ensure math operations are correct
# -time 1ns : All timing reports will display in nanoseconds
# -cap 1pf : All capacitance reports will display in picofarads
set_library_unit -time 1ns -cap 1pf

# Configure the static timing analysis (STA) engine
# ocv : On-Chip Variation (simulates real-world manufacturing differences)
# cpr : Clock Path Pessimism Removal (removes fake delays on shared clock paths)
set_db timing_analysis_type ocv
set_db timing_analysis_cpr both
set_db timing_report_unconstrained_paths true
```

Figure 5: Base Units & Timing Engine Config

5.2.2. Step 2: Loading Inputs and Database Compilation (init_design)

We now point the tool to the global power nets and feed the raw input files (Netlist, LEF, MMMC). The init_design command is the trigger that compiles these disparate text files into a unified, physical, memory-mapped 3D database.

```
# Tell the database what to name the implicit power and ground signals
set_db init_ground_nets $design(all_ground_nets)
set_db init_power_nets $design(all_power_nets)

puts "INFO: Reading MMMC, LEFs, and Genus Netlist..."
read_mmmc $design(mmmc_view_file)
read_physical -lef $tech(ALL_LEFS)
read_netlist $design(postsyn_netlist)

# Compiles the database, builds the timing graph, and initializes the GUI
init_design
```

Figure 6: Importing Design

After running these commands, the output given below which shows the layout:

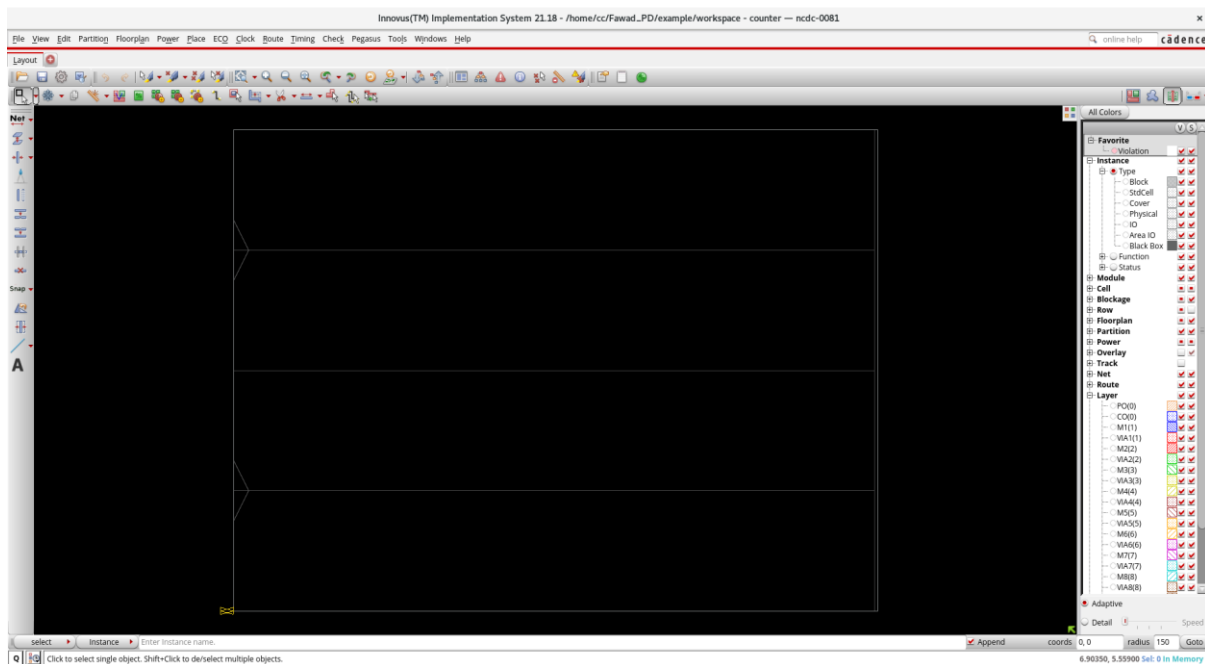


Figure 7: Layout after Importing Design

5.2.3. Step 3: Global Net Connectivity

At this stage, the standard cells are loaded into memory, but their internal power pins (e.g., the microscopic VDD/VSS pins inside a flip-flop) are "floating." We must logically map these physical pins to the global power nets of the chip.

The commands `connect_global_net -type tiehi` and `tielo` are designed to hunt down hardcoded constants in your Verilog netlist.

- If you wrote Verilog code like `(assign enable = 1'b1)`, the synthesis tool leaves that pin floating, waiting for physical design to connect it to a constant "High" voltage.
- The `-type tiehi/tielo` command finds that floating pin and logically wires it to the VDD/VSS net.

```
# Logically link the physical standard cell power pins to the global power nets
# -pin      : The name of the pin on the actual library cell (e.g., "VDD")
# -all      : Apply to all instances in the design
# -verbose  : Print exactly what gets connected to the terminal
connect_global_net $design(digital_gnd) -pin $tech(STANDARD_CELL_GND) -all -verbose
connect_global_net $design(digital_vdd) -pin $tech(STANDARD_CELL_VDD) -all -verbose

# Connect logic 1 and logic 0 tie-offs to VDD and VSS
# -type tiehi/tielo : Specifies that these are constant logic connections
connect_global_net $design(digital_vdd) -type tiehi -all -verbose
connect_global_net $design(digital_gnd) -type tielo -all -verbose
```

Figure 8: Applying Global Nets

After running these commands, the output is given below:



```
@innovus 51> connect_global_net $design(digital_gnd) -pin $tech(STANDARD_CELL_GND) -all -verbose
8 new gnd-pin connections were made to global net 'VSS'.
@innovus 52> connect_global_net $design(digital_vdd) -pin $tech(STANDARD_CELL_VDD) -all -verbose
8 new pwr-pin connections were made to global net 'VDD'.
```

Figure 9: Terminal Output after applying Global Nets

Here 8 pins of VDD/VSS connected to standard cells, you can verify from your design browser i.e

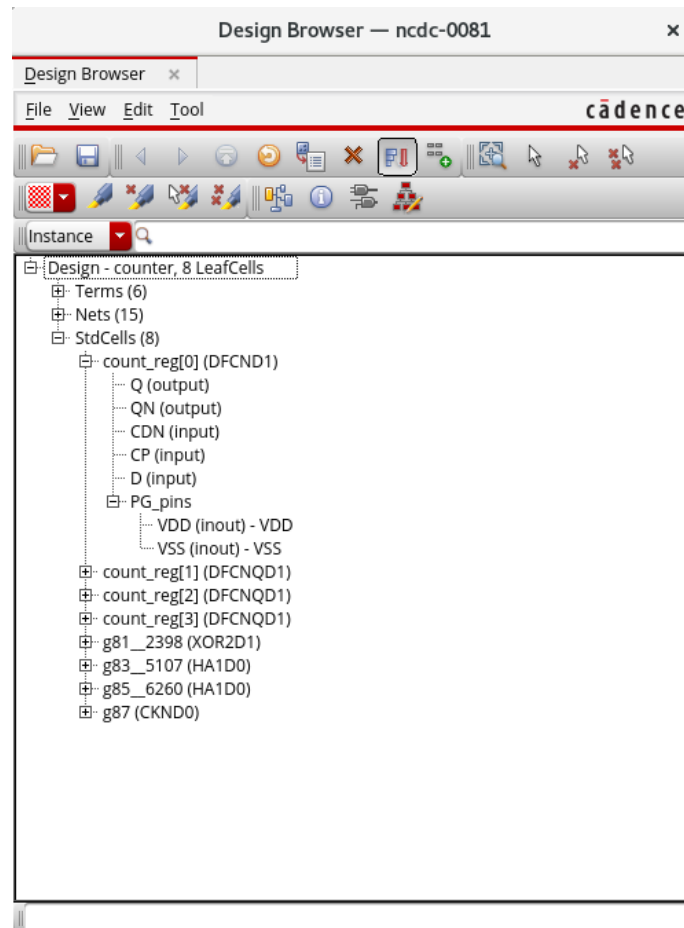


Figure 10: Design Browser

In case for executing the `connect_global_net -type tiehi/tielo` commands, you may observe that the terminal outputs nil or does not list any connections, despite using the `-verbose` flag. This is normal and expected because a standard 4-bit counter contains no static logic constants (1'b1 or 1'b0). Every gate is actively driven by a dynamic signal. Therefore, the tool finds no floating pins requiring a tie-off connection. We leave these commands in the script as a "safety net" best practice, ensuring that if the Verilog code is ever modified to include constants, the physical design script will automatically handle them.

5.3. Floorplanning & Pin Placement

Floorplanning is the first physical step in backend design. In this stage, we define the physical dimensions of the silicon chip, establish the boundary where logic cells are allowed to be placed (the "Core"), and assign locations for the Input/Output (I/O) pins.



5.3.1. Step 1: Defining the Silicon Boundary

We use the `create_floorplan` command to dictate the size and shape of our chip. Instead of guessing absolute X/Y coordinates, we allow Innovus to calculate the size automatically based on our desired Aspect Ratio and Density.

```
# Draw the silicon boundary and the internal standard cell rows
# -flip s      : Flips every alternate row (N-well to N-well) to share power rails
# -match_to_site : Ensures the dimensions snap to legal standard cell grid sizes
# -core_density_size : <AspectRatio(1.0=Square)> <Density(0.6=60%)> <Left> <Bottom> <Right> <Top Margins(um)>
create_floorplan -flip s -match_to_site -core_density_size 1.0 0.6 13.0 13.0 13.0 13.0
```

Figure 11: Floorplan Command

After giving this command, the output is given below:

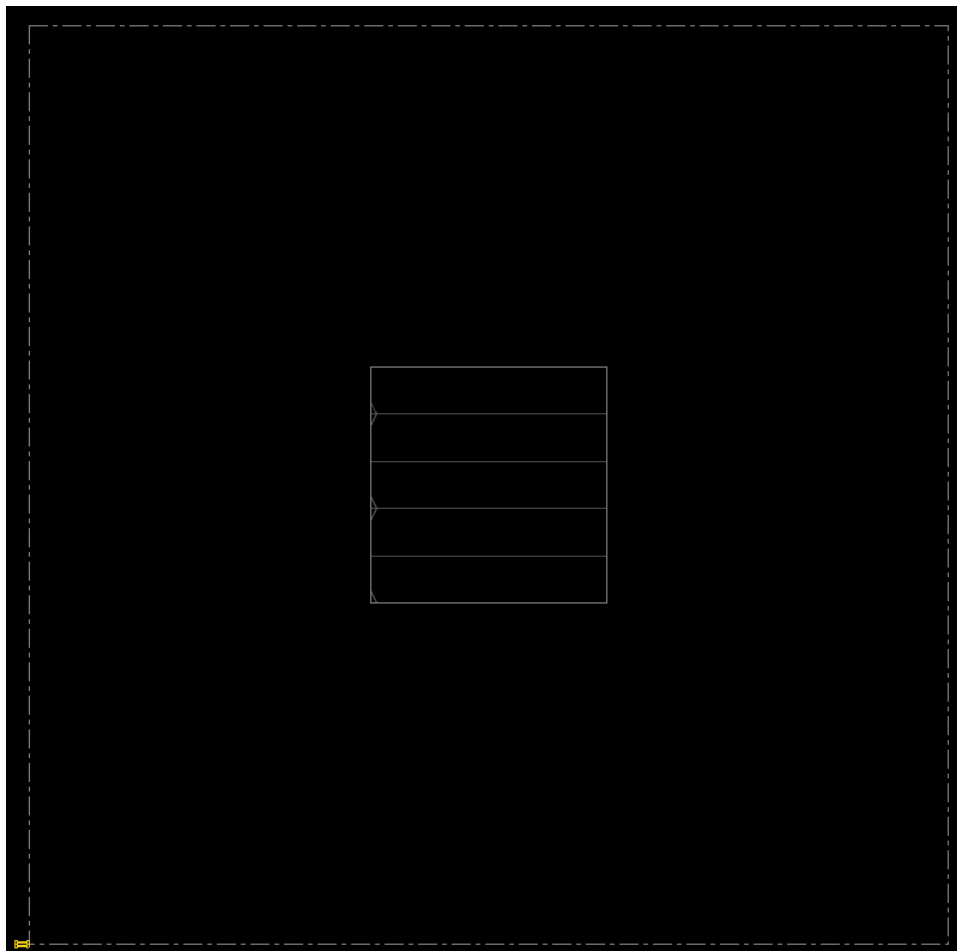


Figure 12: Layout after Floorplanning

As you observed that:

- **Density (0.6):** This means we are packing our logic gates into **60%** of the available core area. The remaining **40%** is left empty! We need this empty space, so the router has room to drop vias and insert clock buffers later in the flow.
- **Margins (13.0 μm):** This is the "safe zone" between the edge of the standard cells and the physical edge of the die. We leave a massive **13.0 μm** gap on all



four sides so we have room to build our massive VDD/VSS power rings in the next step.

- **Row Flipping (-flip s):** Standard cells typically have VDD at the top and VSS at the bottom boundary. By flipping every alternate row upside down, two adjacent rows can share a single VDD or VSS wire, which saves a significant amount of routing space.

Note: To get more information about this command, you should type (man command_name). The manual for create_floorplan is given below:

```
Product Version 21.18 Cadence Design Systems, Inc. create_floorplan(21.18)

Name
create_floorplan - Specifies the floorplan dimensions by size; or by die, I/O, or core coordinates. The create_floorplan command initializes the floorplan and calls the add_tracks
command to create new routing tracks

Syntax
create_floorplan [-help] [-core_margins by {io | die}] [-die_size by io height {min | max}] [-flip {f | s | n}] [-floorplan_origin {center | llcorner}] [-match_to_site] [-no_resize]
[-no_snap_to_grid] [-overlap_same_site_row] [-box_size {<die_box io_box core_box>} | -core_size {<w h left bottom right top>} | -die_size {<w h left bottom right top>} | -core_den-
sity_size {<aspect_ratio>{<row density> {<left bottom right top>}} | -stdcell_density_size {<aspect_ratio> {<std_cell_density> {<left bottom right top>}} | -keep_shape_density
<util>} [-site <name>] | -site_only <name>}

Description
Specifies the floorplan dimensions by size; or by die, I/O, or core coordinates. The create_floorplan command initializes the floorplan and calls the add_tracks command to create
new routing tracks. By default the add_tracks command calculates the optimum spacing between tracks and ignores the pitch values in the tech LEF. When you use the Specify Floor-
plan form (or specify the floorplan through the floorplan command), the floorplan is resized automatically - relatively floorplan constraints are automatically derived on the fly
for blocks, fixed standard cells, fixed pre-routes, and blockages. The floorplan is linearly adjusted as follows:

* Spacing among blocks is evenly adjusted
* The size of the modules and black boxes is evenly adjusted.
* The fixed pre-routes and I/Os are automatically adjusted.

Note: The create_floorplan command checks if the specified IO box and core box is within limits of die box and displays an error message if they are outside the die box.
```

Figure 13: Manual for floorplan command

You should explore and try more options to observe behavior.

5.3.2. Step 2: Placing the I/O Pins

Now that the boundary exists, we must tell the tool where the signals enter and exit the chip. If the I/O file exists, the tool places I/Os according to specified dimensions in the layout, otherwise we use edit_pin command to set I/O pins location manually.

```
if {[file exists $design(io_file)]} {
    # If the file exists, read the exact X/Y coordinates for every pin
    puts "INFO: Custom .io file found! Placing pins using explicit coordinates..."
    read_io_file $design(io_file)
} else {
    puts "WARN: No .io file found. Automatically spreading pins around the boundary..."

    # Place the Clock and Reset evenly on the LEFT side of the block
    # -side Left : Targets the left boundary of the
    # -spread_type center: Centers the group of pins on that edge
    # -layer M4 : Draws the pin shapes using Metal 4
    # -spacing 4 : Leaves 4 tracks of empty space between the pins
    edit_pin -pin {clk} -side Top -spread_type center -layer M4 -spacing 4
    edit_pin -pin {reset} -side Left -spread_type center -layer M1 -spacing 4
    # Place the 4-bit output bus evenly on the RIGHT side of the block
    edit_pin -pin {count[0] count[1] count[2] count[3]} -side Right -spread_type center -layer M1 -spacing 2
}
```

Figure 14: Setting I/O Placement

After giving this block of commands, the output is given below:

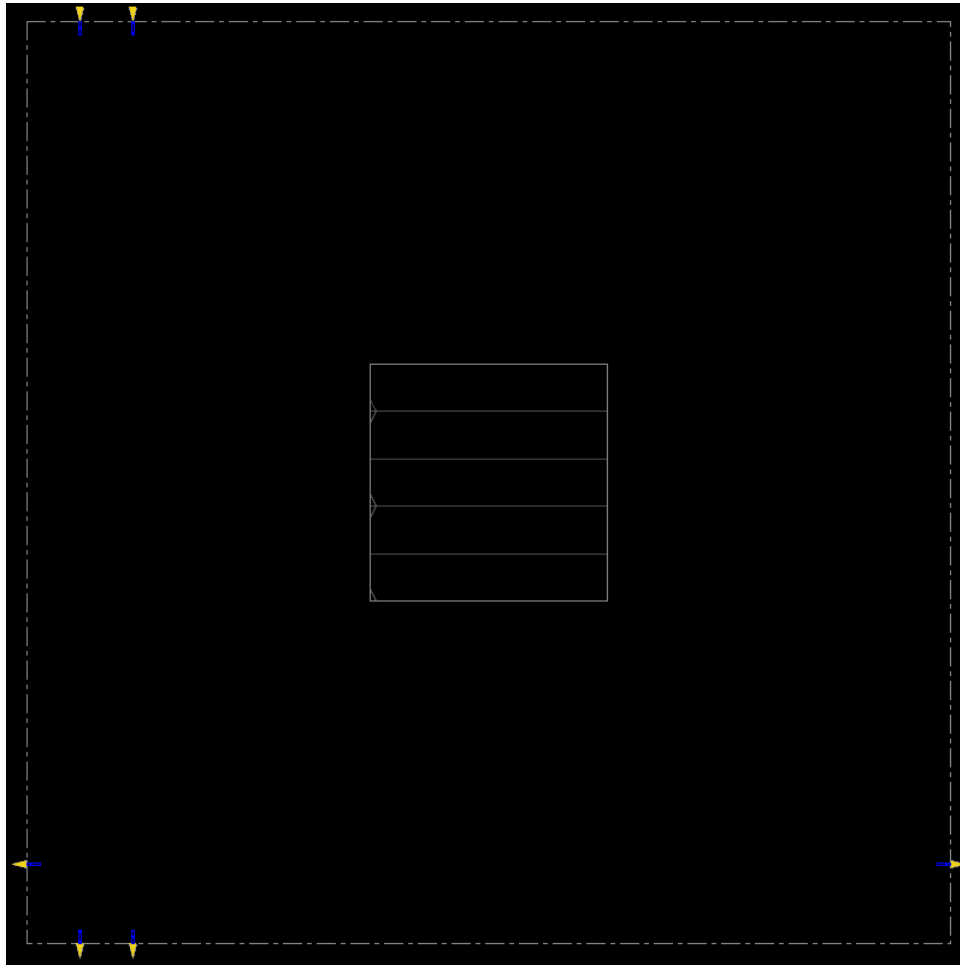


Figure 15: Layout after I/Os Placement

As you observed, pins are placed according to the I/O file. If you want to change the layer or direction, you can run the `edit_pin` command in the `else` section. Try that & observe the output.

5.4. Power-Planning

Before we can place any logic gates, we must build a robust metallic grid to deliver power (VDD) and ground (VSS) to every region of the chip. This structure is known as the Power Delivery Network (PDN). If the PDN is too weak, the chip will suffer from IR Drop (voltage dropping before it reaches the cells), which causes logic gates to slow down or fail completely.

Power planning is done *top-down*, meaning we build massive outer rings first, drop vertical stripes down into the core, and finally connect them to the horizontal standard cell rows.

5.4.1. Step 1: Setting Layers & Core Rings

We begin by dynamically fetching the metal layer names from the database and drawing a massive, highly conductive ring around the perimeter of the core.



```
# Dynamically fetch layer names from the database in case the LEF uses different naming conventions
set m1_name [get_db [lindex [get_db layers] 1] .name]
set m2_name [get_db [lindex [get_db layers] 2] .name]
set m3_name [get_db [lindex [get_db layers] 3] .name]
set m4_name [get_db [lindex [get_db layers] 4] .name]
set m5_name [get_db [lindex [get_db layers] 5] .name]
set m6_name [get_db [lindex [get_db layers] 6] .name]
set m7_name [get_db [lindex [get_db layers] 7] .name]

puts "INFO: Creating Core Rings on M4/M5.."
# Draw a massive, protective ring of VDD/VSS around the core area to eliminate IR Drop
# -type core_rings : Defines the physical object type in the Innovus database
# -nets $design(core_ring_nets) : Links the physical metal to the logical VDD/VSS global nets
# -follow core : Traces the exact boundary of the inner standard cell core
# -layer : Assigns horizontal segments to Top Metal (M7) and vertical to (M6)
# -width : Forces an aggressive 5.4 um thickness for maximum current capacity
# -spacing : Sets a 1.8 um safety gap between VDD and VSS to prevent crosstalk/shorts
# -offset : Pushes the inner ring exactly 0.4 um away from the standard cell Halo
# -center 1 : Balances the remaining space evenly across the routing channel
# -threshold 0 : Ensures all ring segments are drawn regardless of minimum length rules
# -jog_distance 0 : Forces the rings to be perfectly straight (prevents bending around obstacles)
# -snap_wire_center_to_grid none : Prevents Innovus from shifting these massive wires to snap to the microscopic routing grid
add_rings -type core_rings \
-nets $design(core_ring_nets) \
-follow core \
-layer "top $m7_name bottom $m7_name left $m6_name right $m6_name" \
-width {top 5.4 bottom 5.4 left 5.4 right 5.4} \
-spacing {top 1.8 bottom 1.8 left 1.8 right 1.8} \
-offset {top 0.4 bottom 0.4 left 0.4 right 0.4} \
-center 1 \
-threshold 0 \
-jog_distance 0 \
-snap_wire_center_to_grid none
```

Figure 16: Setting Core Rings

After giving these commands, the output is given below:

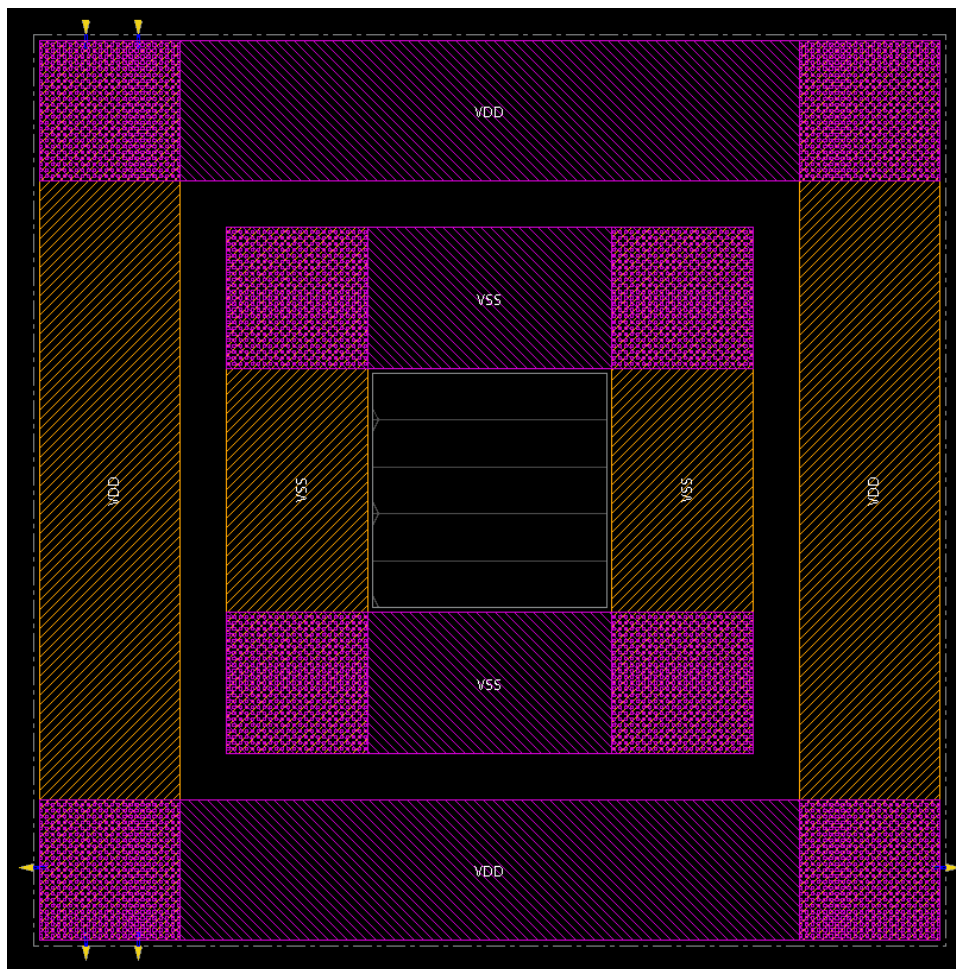


Figure 17: Layout after adding Rings



As you may have observed, the core power ring utilizes Metal 7 (M7) for the top and bottom boundaries, and Metal 6 (M6) for the left and right boundaries. Together, these intersecting metal layers form a continuous protective ring of VDD and VSS around the layout. We use the highest available metal layers (M6/M7) for the main rings because higher metal layers in semiconductor physics are physically thicker and wider, meaning they have the lowest electrical resistance.

5.4.2. Step 2: Follow Pins (Standard Cell Power Rails)

Next, we draw the "local driveways." Standard cells are designed to receive power through horizontal Metal 1 (M1) wires running across the top and bottom of the cell boundaries.

```
# Draws the thin M1 wires that run horizontally across every standard cell row
# -connect {core_pin} : Hooks these wires up to the actual VDD/VSS pins on the std cells
route_special -connect { core_pin } -nets "$design(digital_gnd) $design(digital_vdd)"
```

Figure 18: Power Rails Command

After running this command, the output is given below:

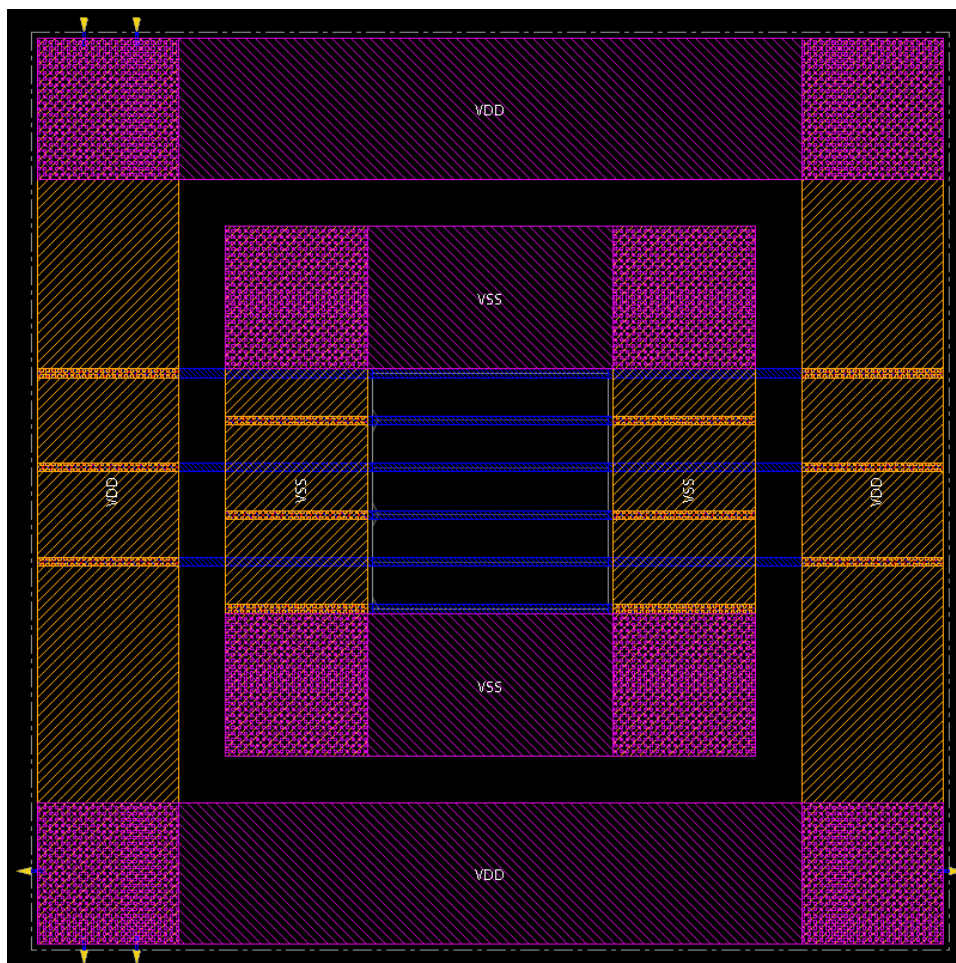


Figure 19: Layout after Power Rails



As you may have observed, the Metal 1 (M1) layer runs horizontally across the core rows to deliver power directly to the VDD and VSS pins of the standard cells."

5.4.3. Step 3: End Caps (Boundary Protection)

Standard cells consist of carefully doped N-wells and P-wells in the silicon substrate. If a row suddenly ends, manufacturing imperfections during the etching process can damage the wells of the outermost logic gates.

```
# Endcaps seal the left and right edges of the standard cell rows to prevent manufacturing defects
set_db add_endcaps_right_edge $tech(ENDCAPS_right)
set_db add_endcaps_left_edge $tech(ENDCAPS_left)
add endcaps -prefix ENDCAP
```

Figure 20: Placing EndCaps Cells

After adding the End Caps cells, the output is given below:

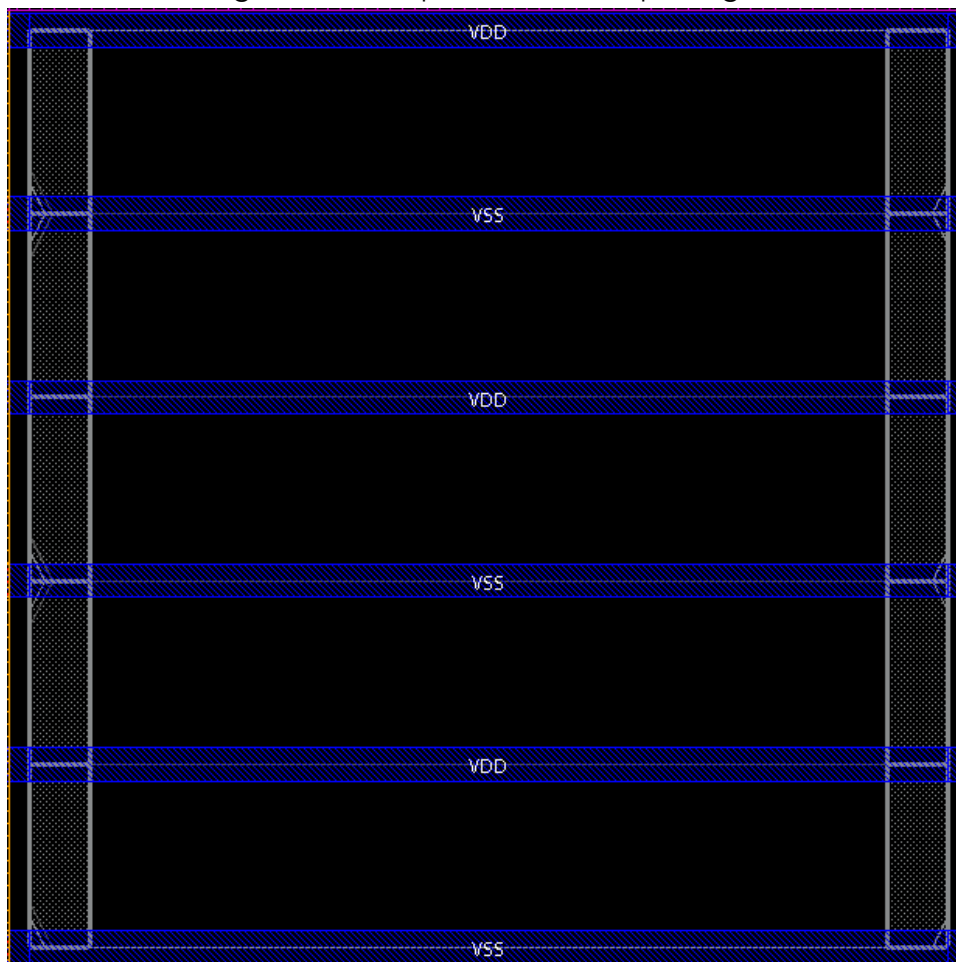


Figure 21: Layout after placing EndCaps Cells

Notice that the left and right boundaries of the standard cell rows are securely sealed with end cap cells. These cells act as protective physical barriers, preventing manufacturing defects at the edges of the silicon well.



5.4.4. Step 4: Vertical Power Stripes

Finally, we must connect the massive M6/M7 outer rings to the thin M1 horizontal rows. We do this by dropping vertical columns (stripes) of M4 across the chip at regular intervals.

```
# Drops vertical columns of power/ground across the chip to feed the M1 rails
# -nets $design(core_ring_nets) : Links physical stripes to logical VDD/VSS power nets
# -direction vertical          : Runs up and down, perpendicular to the horizontal M1 rails
# -layer M4                   : Uses a thicker, intermediate metal for lower resistance
# -width 0.9                   : Sets a robust 0.9 um thickness to prevent voltage drop
# -spacing 0.8                 : Leaves a 0.8 um gap between the VDD and VSS wire in a set
# -set_to_set_distance 4.6     : Automatically drops a new pair every 4.6 um
# -start_from left             : Begins the calculation from the left-hand side of the core
# -start_offset 0.9            : Leaves a 0.9 um gap from the inner ring to avoid DRC shorts
add_stripes -nets "$design(digital_gnd) $design(digital_vdd)" \
  -direction vertical \
  -layer M4 \
  -width 0.9 \
  -spacing 0.8 \
  -set_to_set_distance 4.6 \
  -start_from left \
  -start_offset 0.9
```

Figure 22: Power Stripes Command

After adding power stripes, the output is given below:

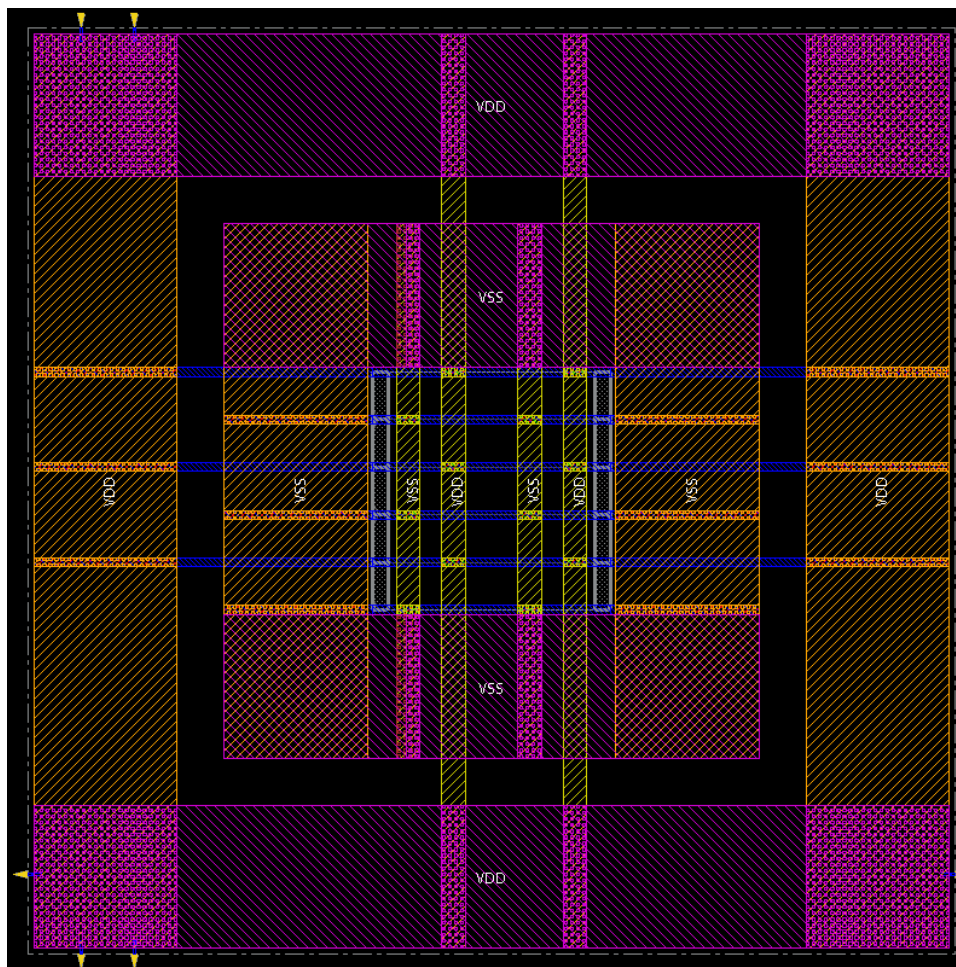


Figure 23: Layout after adding Power Stripes



Notice that the vertical Metal 4 (M4) stripes act as the main distribution of highways within the core. They carry VDD and VSS down from the massive outer rings and distribute the power directly to the horizontal Metal 1 (M1) rails, ensuring every standard cell receives a strong, stable voltage.

Therefore, the power distribution network phase is completed. Moving towards the next phase which is Standard Cells Placement.

5.5. Placement & Pre-CTS Optimization

With the floorplan defined and the power grid constructed, the database is ready to accept the standard cells. During this phase, the tool determines the optimal physical location for every logic gate, ensuring no cells overlap while actively trying to meet your 100MHz setup timing constraints.

5.5.1. Step 1: Congestion-Aware Placement & Optimization

First, we instruct the placement engine to prevent future routing failures by avoiding localized crowding. By setting `place_global_cong_effort` to `auto`, the tool automatically detects areas with high wire density and spreads the standard cells apart to relieve congestion. Next, we execute the `place_opt_design` command. Unlike basic placement, this command actively optimizes the design: it places the cells, estimates preliminary wire delays, and dynamically resizes gates (e.g., swapping a weak XOR2D1 for a stronger XOR2D4) to ensure Setup Timing is met before we proceed.

```
# Tells the placer to automatically spread cells out if it detects high congestion
set_db place_global_cong_effort auto

# Executes placement AND pre-CTS logic optimization
# -report_dir : Saves the timing and congestion metrics to a designated folder
place_opt_design -report_dir "$design(reports_dir)/placement/place_opt_design"

# Verifies that no cells are overlapping and all are snapped to the legal grid
check_place

# Deletes any accidental temporary routes generated by the placer
# to ensure a clean slate for the Clock Tree Synthesis engine
delete_routes -type regular
```

Figure 24: Placement Commands

Once the standard cells are placed, you will see them securely populated inside the core layout. Be sure to carefully review the terminal outputs and the generated placement reports to gather critical information about the design status.

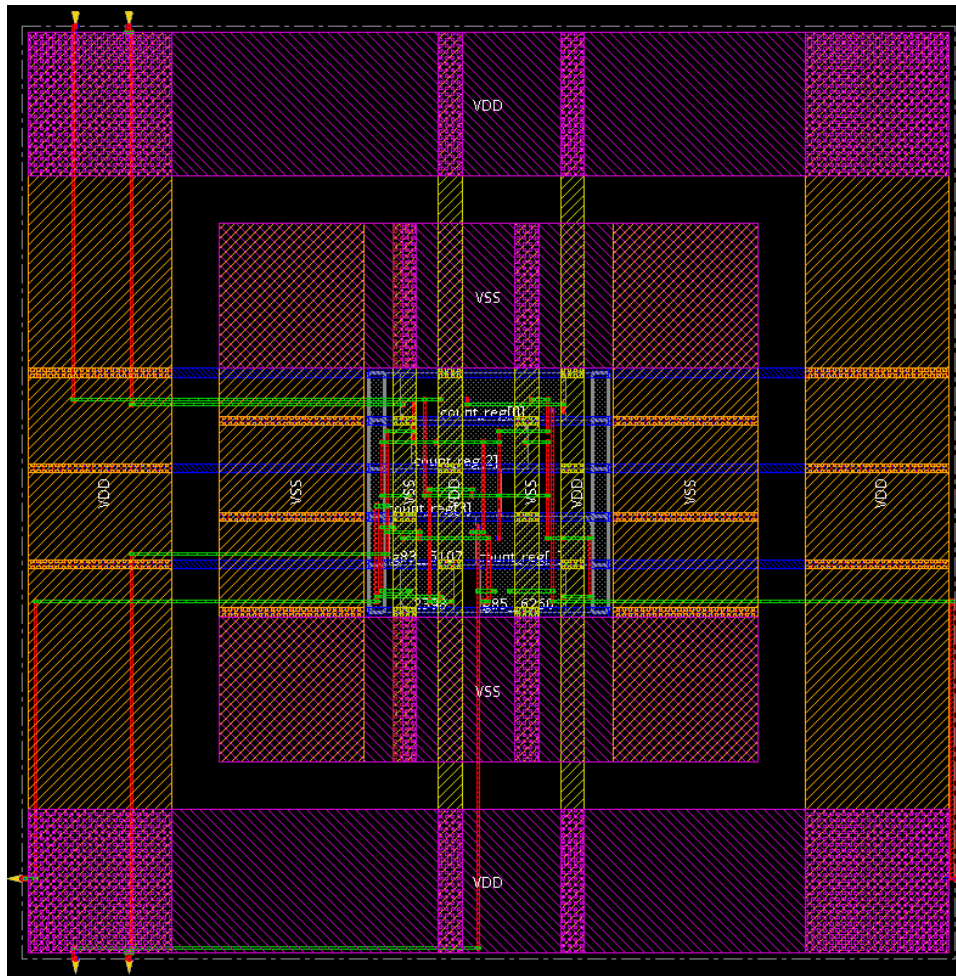


Figure 25: Layout after Placement

As you may have observed, the tool generates rough trial routing during placement to estimate timing. We remove these temporary wires using the `delete_routes` command to prepare for the next stage. Afterward, we use the `check_place` command to verify that all standard cells are placed legally without any overlaps. The output is provided below:

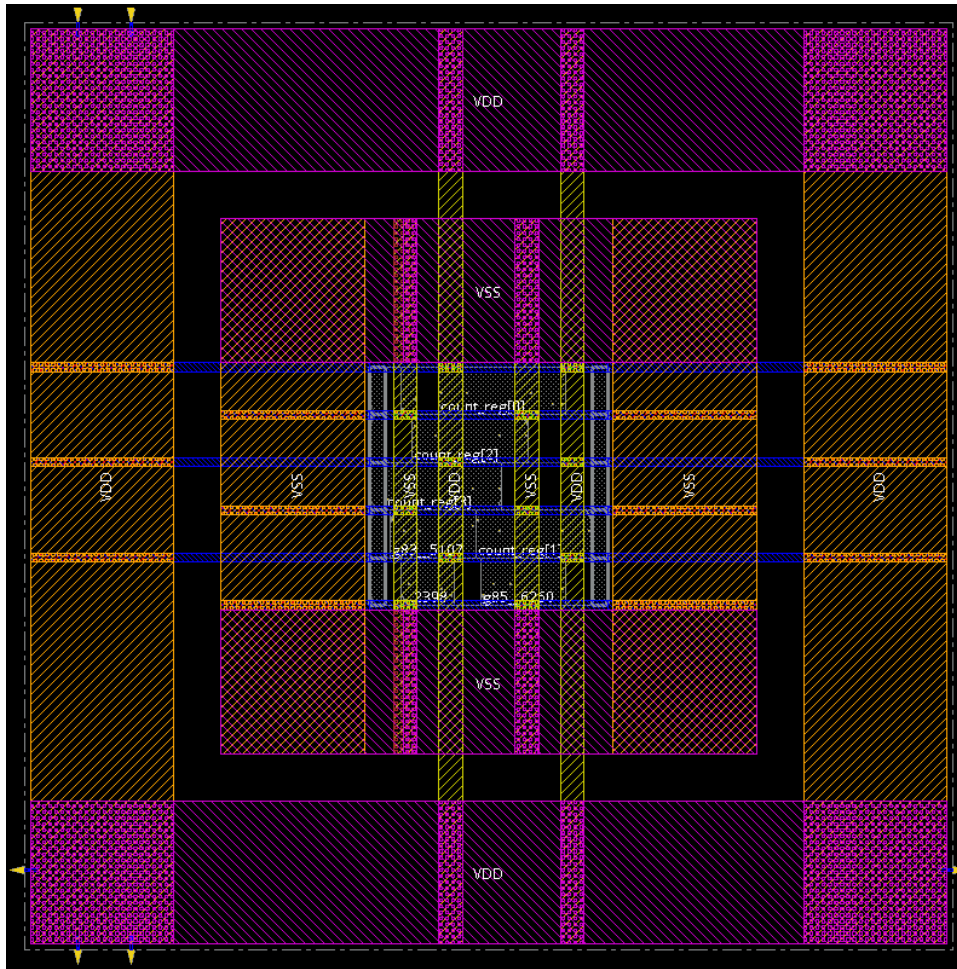


Figure 26: Layout after removing Trial Routing

```
@innovus 167> check_place
Begin checking placement ... (start mem=2567.9M, init mem=2567.9M)
*info: Placed = 18          (Fixed = 10)
*info: Unplaced = 0
Placement Density:69.23%(49/70)
Placement Density (including fixed std cells):73.33%(59/81)
Finished check_place (total: cpu=0:00:00.0, real=0:00:00.0; vio checks: cpu=0:00:00.0, real=0:00:00.0; mem=2567.9M)
0
```

Figure 27: Placement Report

5.5.2. Step 2: Adding Physical Tie Cells

As discussed during the initialization phase, transistor gates cannot be directly wired to the high-voltage VDD or VSS grids. We must drop physical "Tie" cells into the layout to safely step down the current for any constant logic 1s or 0s.

```
# Tie cells act as safe resistors between logic gates and the power grid
set_db add_tieoffs_prefix "TIE"
set_db add_tieoffs_cells "$tech(TIEHI) $tech(TIELO)"
set_db add_tieoffs_max_fanout $tech(TIE_MAX_FANOUT)
add_tieoffs
```

Figure 28: Adding Tie-Cells



Note: As we observed in the initialization logs, this 4-bit counter may report 0 Tie cells added because it contains no hardcoded constants. However, this command is a strict industry requirement for robust scripts.

5.5.3. Step 3: Pre-CTS Design Rule Violation (DRV) Fixing

Before we build the highly sensitive clock tree, we must ensure the "data" paths are electrically perfect. This command forces the tool to scan wires that are too long, have too much capacitance, or transition too slowly, and fixes them by inserting buffers or upsizing cells. For this purpose, we use `opt_design` command by giving `-pre-cts` & `-drv` flags.

```
# A quick optimization pass focusing purely on repairing electrical violations
# -pre_cts : Tells the engine the clock is still "ideal" (zero delay)
# -drv      : Instructs the engine to fix Design Rule Violations (max cap/tran/fanout)
opt_design -pre_cts -drv -report_dir "${design(reports_dir)}/opt/pre_cts"
```

Figure 29: Pre-CTS Command

Be sure to carefully review the terminal outputs and the generated pre-cts reports to gather critical information about the design status.

After pre-CTS optimization, the logic has been physically placed and resized. We must export this state to verify that the placement engine did not break the logic.

```
# --- LEC CHECKPOINT 1: POST-PLACEMENT ---
# Verifies setup-timing fixes and high-fanout buffering did not break the logic
write_netlist ${design(export_dir)}/pnr/${design(TOPLEVEL)}_placed.v
```

Figure 30: Pre-CTS Netlist

5.6. Clock Tree Synthesis (CTS) & Post CTS Optimization

The clock signal is the heartbeat of a digital chip. Clock Tree Synthesis (CTS) is the process of physically routing the clock net from its input pin to the clock pin of every sequential element (flip-flop) in the design. The ultimate goal of CTS is to achieve zero clock skew - meaning the clock edge arrives at every single flip-flop at the exact same picosecond.

5.6.1. Step 1: Defining the Clock Tree Specification

Before Innovus can build the tree, it needs a blueprint. It must analyze your constraints (.sdc) to identify where the clock enters the chip (the "Root") and which flip-flops it needs to connect to (the "Sinks"). For this purpose, we use `create_clock_tree_spec`.

```
# 1. WRITE THE RECIPE
# The tool automatically analyzes your .sdc and identifies clock sources and Flip-Flop sinks
create_clock_tree_spec
```

Figure 31: Creating file for clock_tree_specs

After giving this command, the terminal output is given below:



```
Creating clock tree spec for modes (timing configs): func_mode
cts_spec config_create_generator_skew_groups=true: create_clock_tree_spec will generate skew groups with a name prefix of "_clock_gen" to balance clock generator connected flops with the cl
ock generator they drive.
Reset timing graph...
Ignoring AAE DB Resetting ...
Reset timing graph done.
Ignoring AAE DB Resetting ...
Analyzing clock structure...
Analyzing clock structure done.
Reset timing graph...
Ignoring AAE DB Resetting ...
Reset timing graph done.
Extracting original clock gating for clk...
clock tree clk contains 4 sinks and 0 clock gates.
Extracting original clock gating for clk done.
The skew group clk/func mode was created. It contains 4 sinks and 1 sources.
Checking clock tree convergence...
Checking clock tree convergence done.
```

Figure 32: Terminal Output for Creating Clock Tree Specs

As you may have observed in the CTS log, the clk net is driven by a single source and distributes the signal to exactly four sinks.

5.6.2. Step 2: Building and Balancing the Tree (ccopt_design)

With the specification generated, we launch Cadence's proprietary CCOpt (Clock Concurrent Optimization) engine. This engine works like a plumber installing pipes: it drops specialized clock buffers into the core and connects them with thick copper wires, aggressively balancing the wire lengths and buffer delays to eliminate skew.

```
# 2. COOK THE MEAL
# The Cadence CCOpt engine builds the physical copper tree and balances delays to achieve zero skew
ccopt_design -report_dir "$design(reports_dir)/cts/ccopt_design"
```

Figure 33: Command for Clock Tree Creation

Be sure to carefully review the terminal outputs and the generated ccopt_design reports to gather critical information about the design status.

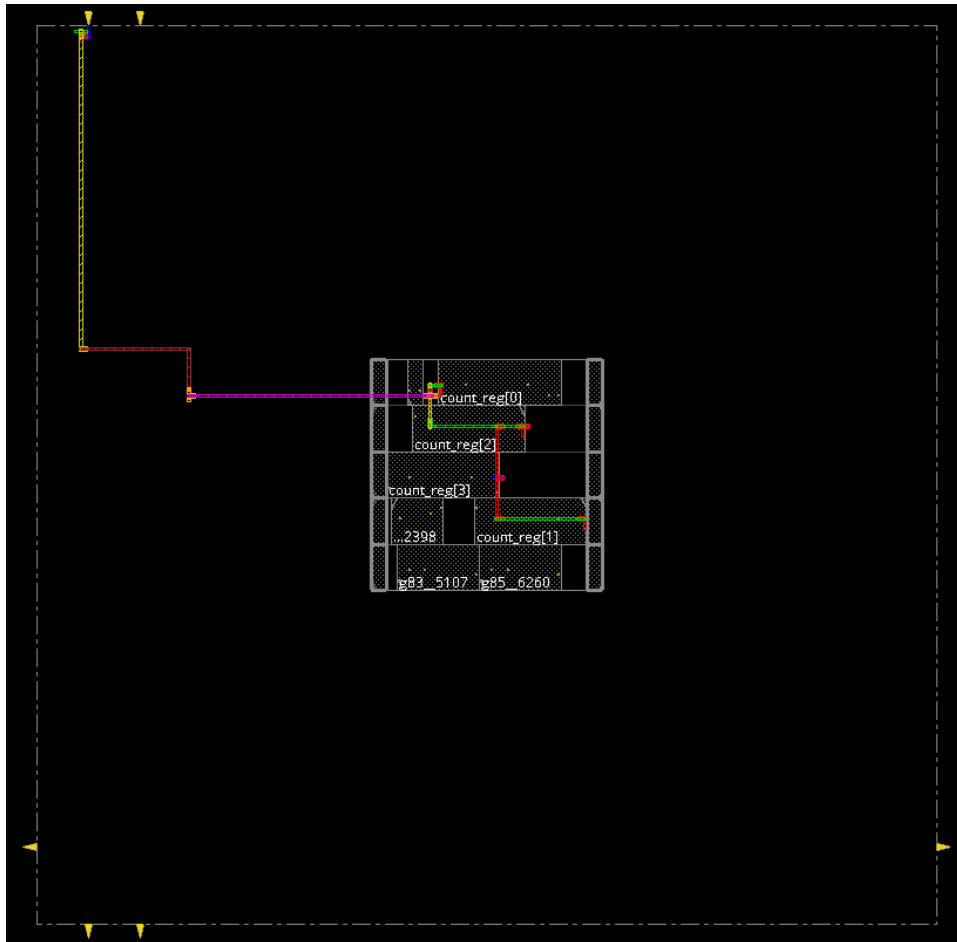


Figure 34: Layout after running CTS Command

5.6.3. Step 3: Post-CTS Optimization

This is a critical turning point in the physical design flow. Because the clock is now made of real physical copper and buffers, it takes time for the signal to travel from the root to the flip-flops (Clock Insertion Delay). Because the clock is now delayed, data signals might arrive at the next flip-flop *too early*, causing a Hold Violation. We must immediately run an optimization pass to fix these ultra-fast data paths by intentionally injecting delay buffers into them.

For this purpose, we use `-post_cts` flag alongwith `setup` & `hold` flags in `opt_design` command. The `-post_cts` flag formally switches the internal timing engine into a Propagated Mode. From this point forward, every timing calculation uses the actual physical delay of the copper clock network.

```
# The clock is now real copper. We must fix data paths that are too fast (Hold violations).  
# -post_cts : Switches the engine from "Ideal" to "Propagated" clock mode  
# -setup -hold -drv : Instructs the engine to fix all constraint types simultaneously  
opt_design -post_cts -setup -hold -drv -report_dir "$design(reports_dir)/opt/post_cts"
```

Figure 35: Post-CTS Command

Be sure to carefully review the terminal outputs and the generated `post_cts` reports to gather critical information about the design status.



CTS violently modifies the design by inserting hundreds of clock buffers. We export this netlist to prove that the clock network substitution is logically sound.

```
# --- LEC CHECKPOINT 2: POST-CTS ---  
# Verifies massive clock buffer insertion did not break the logic  
write_netlist ${design}(export_dir)/pnr/${design}(TOPLEVEL)}_cts.v
```

Figure 36: Post-CTS Netlist

5.7. Signal Routing and Post-Route Optimization

With the clock network fully synthesized and balanced, the final physical task is to route the actual data signals between the logic gates. The Innovus routing engine (NanoRoute) must navigate a highly congested 3D maze of obstacles, dropping vias and changing metal layers to connect every net without causing short circuits or timing violations.

5.7.1. Step 1: Configuring Physics Awareness and ECO Settings

Before launching the router, we must tell it how to route. In sub-micron nodes like TSMC 65nm, wires placed too close together act like capacitors, causing their magnetic fields to interfere with each other (Crosstalk). We instruct the router to actively avoid crosstalk, prioritize the straightest paths for time-critical signals, and prepare for any incremental changes (ECO).

```
# Enable advanced physics awareness and incremental routing for the engine  
# -si_driven : Actively spaces wires apart if electromagnetic crosstalk is detected  
# -timing_driven : Routes critical timing paths as straight as possible to save picoseconds  
# -eco : Enables Engineering Change Order mode to patch specific routes without ripping up the whole chip  
set_db delaycal_enable_si true  
set_db route_design_with_timing_driven true  
set_db route_design_with_si_driven true  
set_db route_design_with_eco true
```

Figure 37: Routing Configuration

5.7.2. Step 2: Global and Detailed Routing (route_opt_design)

Routing happens in two phases. Global Routing divides the chip into a grid and plans rough paths (like a GPS planning a highway route). Detailed Routing assigns the exact metal tracks and drops the physical vias (like driving the actual car). The route_opt_design command handles both simultaneously while running timing checks in the background. For this purpose, we use route_opt_design command.

```
# Interleaves NanoRoute with the timing optimizer  
# (Routes -> Checks timing -> Resizes if needed -> Routes)  
route_opt_design
```

Figure 38: Routing Command

After giving this command, the layout is given below:

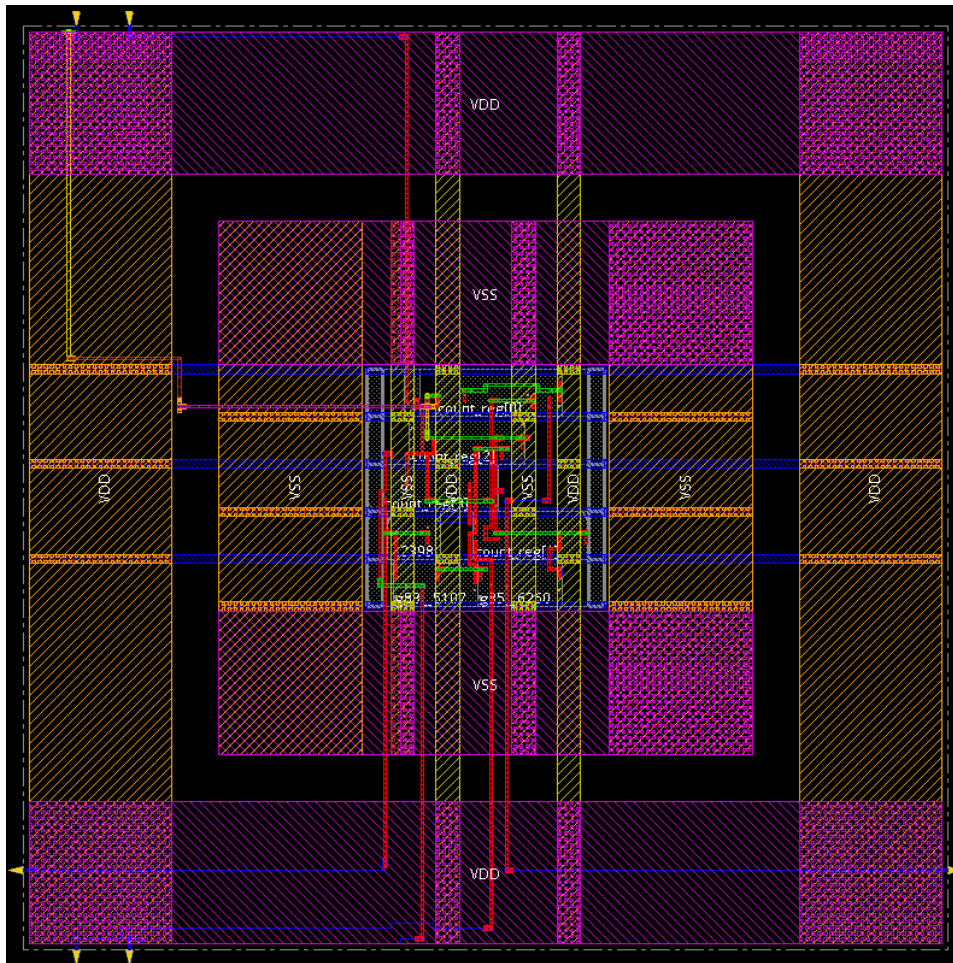


Figure 39: Layout after Routing

5.7.3. Step 3: Post-Route Timing Optimization

Now that the data wires are made of real copper, they possess actual Resistance (R) and Capacitance (C). This physical reality often causes minor timing regressions compared to the estimates made during the Placement phase. We run one final optimization pass to repair any damage. For this purpose, we will use again `opt_design` command with flags `-post_route` along with `setup`, `hold` & `drv` flags.

```
# Final cleanup crew: Fixes any minor timing damage caused by the physical copper routes
opt design -post route -setup -hold -drv -report dir "$design(reports dir)/opt/post route"
```

Figure 40: Post-Route Command

Be sure to carefully review the terminal outputs and the generated post_route reports to gather critical information about the design status.

After NanoRoute fixes hold violations by inserting delay cells, we export the final logic state.

Note: DFM and Filler cells added later do not affect Boolean logic.



```
# --- LEC CHECKPOINT 3: POST-ROUTE ---  
# Verifies final NanoRoute hold-timing fixes and delay cells  
write_netlist ${design}(export_dir)}/pnr/${design}(TOPLEVEL)}_routed.v
```

Figure 41: Post-Route Command

5.8. Design for Manufacturing (DFM)

Even if Innovus reports zero routing violations, the chip might still fail at the TSMC factory. Design for Manufacturing (DFM) is a series of physical optimizations designed to maximize your Yield (the percentage of chips that work after being manufactured).

5.8.1. Step 1: Disabling Routing Engines and Configuring DFM Variables

Because DFM requires shifting physical metal around purely for manufacturing safety, we must temporarily disable the timing, signal integrity, and ECO engines. If we leave them on, they will restrict the tool from making the physical modifications we need. Once disabled, we configure the DFM effort variables.

```
# 1. Disable timing, SI, and ECO engines so the router has full permission to move existing wires  
set_db route_design_with_timing_driven false  
set_db route_design_with_si_driven false  
set_db route_design_with_eco false  
  
# 2. Set DFM effort variables to maximum  
set_db route_design_detail_post_route_spread_wire true  
set_db route_design_detail_use_multi_cut_via_effort high
```

Figure 42: Disabling Routing Configuration

These variables configure the router to actively prevent manufacturing defects (short and open circuits). Setting `route_design_detail_post_route_spread_wire` to true forces the tool to push tightly packed parallel wires further apart into available empty space, minimizing the risk of microscopic dust particles bridging the gap and causing fatal short-circuits.

Simultaneously, setting `route_design_detail_use_multi_cut_via_effort` to high instructs the tool to aggressively replace vulnerable single vias with redundant "multi-cut" (double) vias, ensuring that if one vertical copper connection fails during fabrication, the backup safely carries the current to prevent an open circuit.

5.8.2. Step 2: Multi-Cut Via Insertion and Wire Spreading

With the routing engines disabled and our DFM variables set to maximum effort, we now command the NanoRoute engine to physically modify the layout. These two commands scan the entire database and make thousands of microscopic adjustments to improve the chip's manufacturing yield.

```
# 3. Execute Multi-Cut Via Insertion  
# -via_opt : Replaces vulnerable single-hole vias with redundant 2-hole vias to increase yield  
puts "INFO: Upgrading single vias to multi-cut vias..."  
route_design -via_opt  
  
# 4. Execute Wire Spreading  
# -wire_opt : Pushes tightly packed parallel wires apart to prevent short-circuits from dust/etching  
puts "INFO: Spreading wires to reduce short-circuits..."  
route_design -wire_opt
```



Figure 43: DFM Optimization

For this purpose, we use following commands in which description given below:

- **Multi-Cut Vias (route_design -via_opt):** Replaces vulnerable single vias with redundant "multi-cut" (double) vias wherever space permits. This redundancy ensures that if one via hole fails to form properly during fabrication, the backup hole safely carries the current, preventing a fatal **Open Circuit**.
- **Wire Spreading (route_design -wire_opt):** Actively shifts tightly packed parallel wires into unused routing tracks. Increasing this physical distance prevents fatal **Short Circuits** caused by manufacturing imperfections (like copper widening). As a bonus, this wider spacing reduces parasitic capacitance, which improves signal integrity.

Once the DFM commands finish executing, carefully review the terminal outputs to confirm how many vias were successfully upgraded and verify that the layout remains completely free of routing violations.

5.8.3. Step 3: Restoring the Database State

Notice that immediately after the DFM commands finish executing, we set the timing, signal integrity, and ECO variables back to their original states (true). This is a critical best practice in physical design: you must always return the tool environment to its standard state so that any subsequent commands or timing checks behave normally.

```
# 5. Restore normal routing variables to keep the database safe for future commands
set_db route_design_detail_post_route_spread_wire false
set_db route_design_with_timing_driven true
set_db route_design_with_si_driven true
set_db route_design_with_eco true
```

Figure 43: Restoring Configs of Routing

5.9. Physical Continuity (Filler and Decap Insertion)

After the Placement and Routing phases are complete, your standard cell rows will look like a barcode: there will be clusters of logic gates separated by empty physical gaps. While this is perfectly fine for the logical operation of the chip, it is absolutely illegal for physical silicon manufacturing.

We must achieve **Physical Continuity** by plugging these empty gaps with specialized dummy cells.

5.9.1. The Physics of Continuity (Why do we need fillers?)

Standard cells are built on top of a silicon substrate using carefully doped regions called **N-Wells** and **P-Wells**.

- **Preventing DRC Violations:** Semiconductor foundries (like TSMC) require these wells to be continuous across the entire row. If there is a gap, the well is "broken," which violates minimum area and spacing Design Rules (DRCs).



- Preventing Latch-Up: Broken wells can trigger a parasitic short-circuit effect in the silicon called "Latch-Up," which will literally fry the chip the moment it is powered on.
- Power Rail Continuity: The horizontal Metal 1 (M1) VDD and VSS rails must flow uninterrupted across the row. Gaps break the power delivery to neighboring cells.

Now we will add these cells in the next section.

5.9.2. Inserting Decaps and Fillers

To fix these gaps, we instruct Innovus to fill the empty spaces. Instead of using plain dummy cells for every gap, we prioritize Decoupling Capacitors (Decaps) for the larger holes and use standard Fillers for the tiny holes. For this purpose, we use add fillers command.

```
# Define the prefix name for the new dummy instances
set_db add_fillers_prefix FILLDECAP

# Feed the tool a list of all legal dummy cells (both Decaps and standard Fillers)
# The tool uses a Greedy Algorithm: it plugs the largest holes with Decaps first,
# and then fills the remaining tiny slivers of space with standard Fillers.
set_db add_fillers_cells [concat $tech(DECAP) $tech(FILLERS)]
add_fillers
```

Figure 44: Adding Filler Cells

Once the filler and decap cells are inserted, you must run the route_eco -fix_drc command to resolve any minor Design Rule Check (DRC) violations caused by the new cells. Following this, use the edit_trim_routes -all command to safely remove any unnecessary, dangling wire segments. Finally, execute delete_empty_hinst to clean up the internal database.

```
# Final micro-route to fix any tiny DRC shorts caused by dropping in the filler cells
route_eco -fix_drc
edit_trim_routes -all      ;# Trims away any microscopic dangling pieces of metal
delete_empty_hinst        ;# Cleans up hierarchy by deleting physically empty logical folders
```

Figure 45: DRC Fix

The output after adding cells is given below:

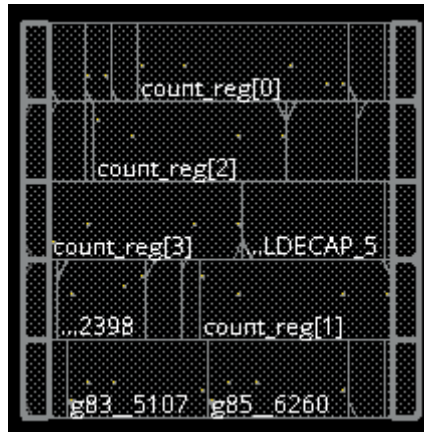


Figure 46: Layout after adding Filler Cells

5.9.3. Standard Fillers vs. Decaps Cells

- **Standard Fillers:** These are completely empty dummy cells. They contain no transistors. Their only job is to provide continuous N-Well, P-Well, and M1 power rails to bridge the gap between two real logic gates.
- **Decap Cells (Decoupling Capacitors):** These are advanced filler cells that act like microscopic, local batteries. In a digital chip, when the clock edge hits, thousands of flip-flops switch simultaneously, drawing a massive spike of current. This sudden draw can cause the VDD voltage to sag (Dynamic IR Drop). Decap cells store a small electrical charge and release it exactly when the neighboring cells switch, stabilizing the local power grid!

5.10. Physical Verification (DRC/Connectivity) and Power Analysis

Before running final timing, signoff, or exporting the database, we must comprehensively audit the physical layout. Even though the router attempts to obey all rules, the complexity of millions of wires means we must run dedicated verification engines to prove the chip has no geometric violations, no broken wires, and a stable power profile.

5.10.1. Design Rule Check (DRC)

The foundry provides a thick manual of manufacturing rules (e.g., "Metal 1 wires must be at least 0.12 μ m apart"). The `check_drc` command scans every polygon in the design to ensure none of these microscopic rules was broken during the routing and DFM phases.

```
# 1. Verify Geometry (DRC)
# Scans the entire layout for spacing, width, and area violations
# -out_file : Saves the violation summary to our designated reports directory
puts "INFO: Running Design Rule Check (DRC)..."
check_drc -out_file "$design(reports_dir)/verify/drc_violations.rpt"
```

Figure 47: DRC



5.10.2. Connectivity Check

It ensures that the physical metal on your screen perfectly matches the logical connections defined in your synthesized netlist. The `check_connectivity` command verifies that there are no broken wires (opens) or accidentally merged wires (shorts).

```
# 2. Verify Connectivity (LVS)
# -type all : Checks for Unrouted nets (Opens), Short-circuits, and Floating pins
# -out_file : Saves the violation summary to our designated reports directory
puts "INFO: Running Connectivity Check..."
check_connectivity -type all -out_file "$design(reports_dir)/verify/connectivity.r
```

Figure 48: LVS

5.10.3. Power Analysis

Before we sign off on the design, we must estimate how much heat the chip will generate and how much current it will draw from the battery. Since we are running this without a simulation testbench, we use Vectorless Analysis, which assumes a statistical "switching activity" for every gate. The commands are given below for this purpose.

```
# 1. Reset Environment
# Clears any leftover power data to ensure a clean, accurate calculation.
set_power_output_dir -reset
set_default_switching_activity -reset
read_activity_file -reset
set_power -reset

# 2. Configure Switching Activity
# -input_activity 0.2 : We assume logic gates flip states (0 to 1) 20% of the time
# -period 10.0 : The analysis is synced to our 100MHz clock (10.0ns period)
set_default_switching_activity -input_activity 0.2 -period 10.0

# 3. Generate the Final Power Report
# -rail_analysis_format VS : Formats the data for IR Drop analysis (Voltus).
report_power -rail_analysis_format VS -out_file "$design(reports_dir)/verify/power
```

Figure 49: Power Analysis

After getting power reports, we get three categories:

- **Leakage Power:** The "always-on" power. Even if the clock is stopped, transistors "leak" a tiny amount of current. This is primarily a function of the fabrication process and temperature.
- **Internal Power:** The energy consumed inside the standard cell itself (e.g., the power required to flip internal nodes of a Flip-Flop). This also accounts for the "short-circuit" current that flows when both P and N transistors are momentarily active during a logic transition.



- **Switching Power:** The energy required to charge and discharge the capacitance of the copper wires we just routed. Longer wires = higher capacitance = higher switching power.

So, total power is a summation of these three categories.

Following the DRC and connectivity checks, audit the results for any remaining violations. If errors persist, use the `opt_design -post_route` command to perform a final cleanup. This command is designed to fix physical and timing violations that only become apparent after the detailed copper routing is finalized.

5.11. Backend Formal Verification (LEC)

While Innovus builds the physical chip, we must use Cadence Conformal to ensure the math remains correct. In backend design, we do not run LEC *inside* the Innovus script. Instead, we export intermediate netlists and run a standalone Conformal script in the Linux terminal.

The Golden vs. Revised Rule for Backend:

- **Golden Design:** Your completely verified, optimized netlist from Front-End Synthesis (e.g., `counter_physical_synth.v`).
- **Revised Design:** The intermediate netlists you just exported from Innovus (`_placed.v`, `_cts.v`, or `_routed.v`).

Unlike Genus, Cadence Innovus does not automatically generate a Conformal .do script. Therefore, we have created a master .do file located in the scripts folder. To run this verification, execute the following command in your Linux terminal.

```
@innovus 1> lec -lp -dofile /home/cc/Fawad PD/RISC V FE/scripts/Conformal SPT.do -nogui
```

Figure 50: LEC Initialization

Before running each check, you must update the file path for the -revised design in your script to match the specific netlist you are verifying. After executing the command, analyze the Conformal output log using the same verification metrics introduced in the Synthesis lab.

5.12. Signoff Final STA

After the physical layout is finalized, including the addition of filler cells and DFM optimizations, we must perform a high-fidelity timing audit. Earlier in the flow, the tool used "Fast Timing" estimates to speed up placement. Now, we invoke the Cadence Tempus engine to perform a rigorous signoff analysis using the actual physical properties of the copper wires.

By using the `-post_route` flag, we tell the timing engine to stop using estimates and instead use the **Propagated Clock**. This means the tool calculates the exact delay of every buffer and wire segment in the clock tree we built in Phase 6.



```
# Enable Signal Integrity (SI) to account for crosstalk during the final audit
set_db delaycal_enable_si true

# 1. Signoff Setup Analysis
# Analyzes the "Slow" corner to ensure the chip hits its target frequency (100MHz)
puts "INFO: Running Signoff Setup Analysis..."
time_design -post_route -report_dir $design(reports_dir)/signoff/setup

# 2. Signoff Hold Analysis
# Analyzes the "Fast" corner to ensure data doesn't "race" through flip-flops too
puts "INFO: Running Signoff Hold Analysis..."
time_design -post_route -hold -report_dir $design(reports_dir)/signoff/hold
```

Figure 51: Final STA

Open the generated reports in your signoff directory. You are looking for a metric called Slack which covers every path:

- **Positive Slack (+):** The design meets the timing requirement. The signal arrived earlier than the deadline.
- **Negative Slack (-):** The design fails. You have a "Timing Violation" that must be fixed before the chip can be manufactured.

5.13. Final Design Export (Handoff Deliverables)

At this stage, the design is physically complete, timing is closed, and all verification checks have passed. We now export the database into standard industry formats. These files act as the "contract" between the designer and the manufacturer.

5.13.1. Step 1: Exporting the Netlist and Parasitics

The netlist and parasitic are used by the verification team to perform independent timing analysis.

```
# write_netlist : Saves the final gate-level connectivity (used for LVS/GLS)
# -exclude_leaf_cells : Removes standard cell internal logic to keep file size small
write_netlist ${design(export_dir)}/pnr/${design(TOPLEVEL)}.final.v -exclude_leaf_cells

# extract_rc : Calculates wire resistance/capacitance based on final routing geometry
# write_parasitics : Generates the .spef file used by external timing analyzers
extract_rc
write_parasitics -spef_file ${design(export_dir)}/pnr/${design(TOPLEVEL)}.spef
```

Figure 52: Exporting Netlist & Parasitics

5.13.2. Step 2: Generating the GDSII (The "Tapeout" File)

The GDSII file is the most important deliverable. It is a binary database containing every single geometric polygon on every layer of the chip. This is the file the foundry uses to create lithography masks.



```
# write_stream: Generates the ultimate GDSII mask layout file to send to the fabrication foundry (TSMC)
# -merge: Pointed to the /gds/ sub-folder exactly as shown in the file tree.
# -map_file: Pointed to the /map/ sub-folder exactly as shown in the file tree.
write_stream ${design(export_dir)}/pnr/${design(TOPLEVEL)}.gds \
  -unit 1000 \
  -mode ALL \
  -merge "${WORK_DIR}/libraries/gds/tc6n65lp.gds" \
  -map_file "${WORK_DIR}/libraries/map/PRTF_ED1_N65_gdsout_6X1Z1U.24a.map"
```

Figure 53: Exporting GDSII

Here is the final layout after performing all steps

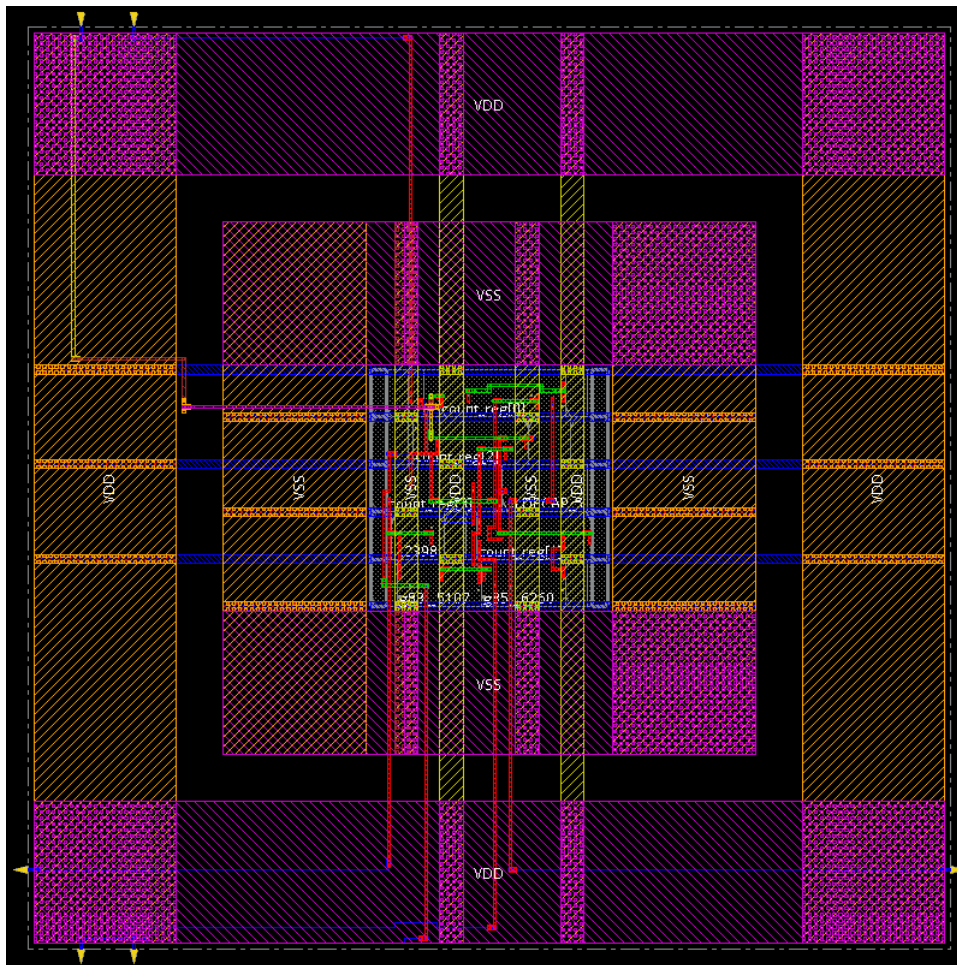


Figure 54: Final Layout

5.14. Physical Verification (LVS Signoff)

Even though the Innovus router checks connectivity and Design Rule Check (DRC) violations internally, the semiconductor foundry requires a completely independent, mathematically rigorous audit before manufacturing. **Layout Versus Schematic (LVS)** is the ultimate physical signoff check.

During this process, Cadence PVS (Physical Verification System) scans the exported geometric polygons from the GDSII file, figures out what transistors and wires they form, and compares this extracted physical circuit against the logical Verilog netlist. If they match perfectly, the layout is mathematically proven to be identical to the intended logic. Here are the files needed:



5.14.1. The LVS Control File (.ctl)

To run PVS, we use a Control File (.ctl). This file acts as a configuration script, telling the LVS engine where to find the GDSII layout, the Verilog netlist, the SPICE library (for transistor-level definitions), and how to handle physical-only filler cells.

Ensure the paths and module names (counter) are updated for your specific design:

```
// =====  
// MASTER PVS LVS CONTROL FILE (.ctl)  
// Design: counter  
// =====  
  
// 1. GENERAL LVS SETTINGS & DEBUGGING  
LVS_REPORT_FILE "lvs.rep";           // Specifies the name of the final LVS text report file.  
LVS_REPORT_MAX 1000;                 // Stops reporting after 1000 errors to prevent massive log files.  
LVS_FIND_SHORTS yes;                 // Instructs the tool to find short circuits between Power and Ground.  
LVS_EXPAND_CELL_ON_ERROR no;         // Keeps the error log hierarchical and easier to debug.  
LVS_RECOGNIZE_GATES -none;           // Disables transistor-level gate recognition (ideal for digital cell-based flows).  
  
// 2. PORT & TEXT RULES  
TEXT_DEPTH -primary;                 // Restricts text label extraction to the top-level cell only.  
LVS_IGNORE_PORTS yes;                // Ignores missing physical pin shapes on power/ground nets if routing is correct.  
LVS_COMPARE_PORT_NAMES yes;          // Strictly enforces that Layout port names match Schematic port names.  
  
// 3. VIRTUAL CONNECTIONS (Power/Ground Netting)  
VIRTUAL_CONNECT -colon yes;           // Logically connects disconnected power nets sharing a prefix (e.g., VDD:1 and VDD:2).  
VIRTUAL_CONNECT -semicolon_as_colon yes;  
VIRTUAL_CONNECT -report no;  
VIRTUAL_CONNECT -depth -primary;  
  
// 4. LAYOUT SETTINGS (GDSII - Physical Design)  
LAYOUT_FORMAT gdsii;  
LAYOUT_PRIMARY "counter";             // The top module name in the physical layout.  
LAYOUT_PATH "/home/cc/Fawad_PD/example/export/Innovus_Data/outputs/pnr/counter.gds";  
  
// 5. SCHEMATIC SETTINGS (Netlist / Logical Design)  
SCHEMATIC_PRIMARY "counter";          // The top module name in the logical netlist.  
SCHEMATIC_PATH "/home/cc/Fawad_PD/example/export/Innovus_Data/outputs/pnr/counter.final.v" verilog;  
SCHEMATIC_PATH "/home/cc/Fawad_PD/example/libraries/spi/tc65lp_200a.spi" spice; // SPICE definitions for standard cells.  
  
// 6. PHYSICAL CELLS FILTERING (Dummy, Decap, Fillers)  
// These cells exist ONLY in the physical layout to prevent manufacturing defects.  
// Since they do not exist in the Verilog logic, we must instruct LVS to delete/ignore them.  
LVS_DELETE_CELL DCAP -layout;  
LVS_DELETE_CELL DCAP2 -layout;  
LVS_DELETE_CELL DCAP3 -layout;  
LVS_DELETE_CELL DCAP4 -layout;  
LVS_DELETE_CELL DCAP8 -layout;  
LVS_DELETE_CELL DCAP10 -layout;  
LVS_DELETE_CELL DCAP16 -layout;  
LVS_DELETE_CELL DCAP32 -layout;  
LVS_DELETE_CELL DCAP64 -layout;  
  
LVS_DELETE_CELL FILL1 -layout;  
LVS_DELETE_CELL FILL2 -layout;  
LVS_DELETE_CELL FILL3 -layout;  
LVS_DELETE_CELL FILL4 -layout;  
LVS_DELETE_CELL FILL8 -layout;  
LVS_DELETE_CELL FILL10 -layout;  
LVS_DELETE_CELL FILL16 -layout;  
LVS_DELETE_CELL FILL32 -layout;  
LVS_DELETE_CELL FILL64 -layout;
```

Figure 55: Control File (.ctl) for LVS

5.14.2. The Physical Verification Rule File (.pvl)

While the control file (.ctl) tells the tool where to find your specific design data, the **.pvl (Physical Verification Language)** rule file tells the tool how to analyze it.

The .pvl file is a highly complex, proprietary rule deck provided directly by the semiconductor foundry (e.g., TSMC). An exported GDSII layout is essentially just a collection of "dumb" geometric shapes (rectangles and polygons). The .pvl file



contains the strict mathematical and geometric algorithms required to reverse-engineer these shapes back into a working electrical circuit.

It instructs the Cadence PVS engine on how to:

- **Extract Devices:** Recognize that a specific overlapping arrangement of Polysilicon and N-Well shapes mathematically forms a PMOS transistor.
- **Trace Connectivity:** Understand which metal layers connect to each other through specific enclosures, allowing the tool to correctly trace an entire net from a driver gate to a receiver gate.

Note: Because this file contains highly guarded manufacturing parameters and electrical rules of the TSMC 65nm process, physical design engineers never write or modify the .pvl file. You simply provide it as an input argument when launching the LVS tool.

5.14.3. Executing the LVS Check

Unlike earlier stages, LVS is run outside of Innovus directly from the standard Linux terminal.

Note: The commands below use /home/cc/Fawad_PD/example/... as an example workspace. You **must** replace this path with the actual absolute path of your own project directory before executing them in the terminal.

First, ensure that your LVS output directory exists. If it does not, create it by replacing the placeholder with your actual path:

```
mkdir -p /<your_actual_workspace_path>/export/LVS_Results
```

Figure 56: Creating Run Directory

Next, execute the Cadence PVS LVS engine by providing the control file and the foundry rule deck (.pvl). Notice that the control file and rule file are fetched from the inputs directory, while the results are saved in the newly created LVS_Results directory:

```
/home/cc/mnt/PVS231/tools.lnx86/bin/pvs -lvs \  
-control /<your_actual_workspace_path>/inputs/pegasus_ctrl.ctl \  
-run_dir /<your_actual_workspace_path>/export/LVS_Results \  
/<your_actual_workspace_path>/inputs/lvs.pvl
```

Figure 57: Invoking the PVS LVS Engine

5.14.4. Analyzing the LVS Results and Reports

Once the Cadence PVS engine finishes execution, navigate to your LVS_Results directory. The tool generates several files to document the extraction, Electrical Rule Checks (ERC), and comparison processes. You must carefully review the following critical files to determine if your chip is ready for tapeout:



```
[cc@ncdc-0081 LVS_Results]$ tree
.
|-- counter.lmap
|-- counter.ports
|-- counter.softchk
|-- db2gds_erc.db.log
|-- erc.db
|-- erc.sum
|-- lvs.rep
|-- lvs.rep.cls
|-- lvs.rep.cps
|-- lvs.rep.shorts
|-- matched
|-- pvs_file.index
|-- qaLog.txt
`-- svdb
```

Figure 58: Run Directory

5.14.4.1. The Main Report File (lvs.rep)

This is the primary scorecard of your physical verification. It contains the final verdict of the comparison between your GDSII layout and Verilog netlist.

- **What to look for:** Scroll to the summary section. You are looking for the exact phrase "Matches".
- **Component & Net Counts:** The report lists the total number of Nets, Instances (cells), and Ports. Verify that the "Incorrect" count is exactly 0 for both the schematic and the layout.

5.14.4.2. The Detailed Error Classification (lvs.rep.cls)

If your main lvs.rep file reports a mismatch, this file is your primary debugging tool.

- **What to look for:** It categorizes exactly what failed (e.g., missing instances, extra nets, or pin swapping). It provides the specific net names and cell instances that the tool could not reconcile between the layout and the schematic, allowing you to trace the error back in the Innovus GUI.

5.14.4.3. The Shorts Report (lvs.rep.shorts)

Short circuits (where two different electrical signals accidentally touch) are the most common reason for LVS failure in physical design.

- **What to look for:** If the tool detects that VDD and VSS are bridged, or two data routing tracks overlap improperly, it isolates those specific errors here. It provides the X/Y coordinates of the shorts to help you quickly trim or reroute them.



5.14.4.4. The Electrical Rule Check Summary (erc.sum)

Before comparing the logic, the tool performs basic sanity checks on the electrical properties of the extracted physical design.

- **What to look for:** This file highlights critical electrical failures like floating nets (wires connected to nowhere), floating gates (inputs of standard cells left unconnected), or missing N-Well/P-Substrate tie-off connections.

5.14.4.5. The Soft Check Report (counter.softchk)

This file logs issues related to "soft connections" in the substrate and wells.

- **What to look for:** It helps identify if your VDD or VSS power grid is weakly connected to the standard cell wells, which is critical for preventing Latch-Up DRC/ERC violations.

Note: For Final Signoff Criteria, if your lvs.rep file indicates a perfect match and your erc.sum reports zero severe electrical violations, your physical design has successfully passed LVS verification. The geometric layout is strictly identical to your logical intent and is electrically sound.

5.15. Signoff STA (Tempus)

Once these files are exported, the final step in the tapeout flow is **Signoff**. We move out of the Innovus implementation environment and into **Cadence Tempus**. Tempus is a golden-standard timing analyzer that uses advanced algorithms to verify that the chip will meet its performance targets across all manufacturing corners.

5.15.1. Invoking Tempus

For this step, we use the command `tempus -stylus` to invoke the tool.

```
[cc@ncdc-0081 workspace]$ tempus -stylus

Cadence Tempus(TM) Timing Solution.
Copyright 2022 Cadence Design Systems, Inc. All rights reserved worldwide.

Version:      v22.15-s092_1, built Fri Mar 1 14:05:47 PST 2024
Options:      -stylus
Date:         Thu May 14 09:59:25 2026
Host:         ncdc-0081 (x86_64 w/Linux 4.18.0-372.32.1.el8_6.x86_64) (8cores*16cpus*12th Gen Intel(R) Core(TM) i7-12700 25600KB)
OS:           Red Hat Enterprise Linux 8.6 (Ootpa)

License:

[09:59:25.217073] Configured Lic search path (22.01-s003): 5280@10.19.32.90

tpsx1 Tempus Timing Signoff Solution XL      22.1 checkout succeeded
16 CPU jobs allowed with the current license(s). Use set_multi_cpu_usage to set your required CPU count.

Change the soft stacksize limit to 0.2%RAM (127 mbytes). Set global soft_stack_size_limit to change the value.
environment variable TMPDIR is '/tmp/ssv_tmpdir_243355_vVA5hK'.
#@ Processing -execute option
@tempus 1>
```

Figure 59: Invoking Tempus

5.15.2. Engine Configuration & Data Loading

Before analyzing timing, we must configure the engine to account for the physical realities of 65nm manufacturing. This includes On-Chip Variation (OCV) and Signal Integrity (SI), which accounts for crosstalk interference between parallel wires.



```
# 1. Enable OCV and CPPR (Removes pessimism from shared clock paths)
set_db timing_analysis_type ocv
set_db timing_analysis_cppr both

# 2. Enable Signal Integrity (Essential for 65nm and below)
set_db delaycal_enable_si true

# 3. Load MMMC, LEF, and the Final Routed Netlist
read_mmmc /home/cc/Fawad_PD/example/inputs/mmmc_master.view
read_physical -lef /home/cc/Fawad_PD/example/libraries/lef/tcbn65lp_9lmT2.lef
read_netlist /home/cc/Fawad_PD/example/export/outputs/pnr/counter.final.v
init_design
```

Figure 60: Importing Design

5.15.3. Loading Parasitics (SPEF)

This is the most critical step of standalone STA. We "back-annotate" the actual Resistance and Capacitance (RC) values of the physical copper wires into the timing graph. This replaces the estimated delays with real-world physics.

```
# =====
# 3. LOAD PHYSICAL PARASITICS (SPEF)
# =====
# We map the physical copper delays to their respective timing corners
set spef_file "/home/cc/Fawad_PD/example/export/outputs/pnr/counter.spef"

puts "INFO: Mapping SPEF parasitics to RC Corners..."

# Worst-Case (Setup) Corner
read_spef -rc_corner rc_worst $spef_file

# Best-Case (Hold) Corner
read_spef -rc_corner rc_best $spef_file

# Typical Corner (Optional)
read_spef -rc_corner rc_typical $spef_file
```

Figure 61: Importing SPEF File

5.15.4. Generating Signoff Scorecards

Finally, we generate reports that the design lead uses to approve the chip for tapeout. We audit four critical areas: **Summary**, **Setup**, **Hold**, and **DRVs**.



```
# Define which views to use for Setup and Hold auditing
set_analysis_view -setup {view_func_setup} -hold {view_func_hold}

# Force a full timing update to propagate parasitics through the graph
puts "INFO: Building final timing graph with parasitics..."
update_timing -full

# Create report directory
file mkdir /home/cc/Fawad_PD/example/export/reports/tempus_signoff

# --- Generate Signoff Reports ---
puts "INFO: Generating Signoff Scorecards..."

# 1. Global Summary (Check TNS/WNS here)
report_timing_summary > /home/cc/Fawad_PD/example/export/reports/tempus_signoff/summary.rpt

# 2. Detailed Setup Report (Top 10 Critical Paths)
report_timing -late -max_paths 10 > /home/cc/Fawad_PD/example/export/reports/tempus_signoff/setup_paths.rpt

# 3. Detailed Hold Report (Top 10 Critical Paths)
report_timing -early -max_paths 10 > /home/cc/Fawad_PD/example/export/reports/tempus_signoff/hold_paths.rpt

# 4. Design Rule Violations (Max Cap/Tran/Fanout)
# In Stylus, we explicitly list the DRV types we want to audit
report_constraint -drv_violation_type {max_capacitance max_transition max_fanout} > /home/cc/Fawad_PD/example/export/reports/tempus_signoff/drv.rpt
```

Figure 62: Exporting Timing Information

Here we talk about two important flags -late & -early

- **-late (Setup):** Analyzes the "Slow" corner (Low Voltage, High Temp). We check if the data arrives before the clock edge.
- **-early (Hold):** Analyzes the "Fast" corner (High Voltage, Low Temp). We check if the data stays stable long enough after the clock edge.

When reviewing the summary.rpt, you are looking for the following:

- **WNS (Worst Negative Slack)**
- **TNS (Total Negative Slack)**
- **DRV Violations**

If all these values are met, the design is **Timing Closed** and ready for the foundry!

6. Lab Tasks

Take the provided counter RTL through the complete physical design flow. Your goal is to achieve Zero DRC/Connectivity violations, a clean LVS Match (zero incorrect nets/instances), and Timing Closure (WNS/TNS) in the standalone Tempus signoff environment.

Task 1: Implementation & Layout Generation (Innovus)

Using the provided innovus script, execute the implementation flow one by one. You are responsible for monitoring the tool's progress and ensuring each stage completes successfully.

- **Floorplanning & Power:** Initialize the design and verify that the power rings and stripes are correctly placed.
- **Placement & CTS:** Perform cell placement and synthesize the clock tree. Observe the Clock Tree Summary to ensure the skew is minimized.



- **Routing & DFM:** Complete the signal routing. Apply DFM optimizations (wire spreading and multi-cut via insertion) to increase manufacturing yield.
- **Physical Continuity:** Add filler and decap cells to ensure substrate continuity.

Task 2: Physical & Electrical Verification

Before moving to Signoff, you must audit your layout for physical and electrical health:

- **DRC & Connectivity:** Run `check_drc` and `check_connectivity`. You must resolve all violations. If "Shorts" or "Opens" exist, use `opt_design -post_route` or manual refinement to fix them.
- **Power Analysis:** Perform a vectorless power analysis and record the Total Power (Leakage + Dynamic).

Task 3: Backend Formal Verification

Execute **Cadence Conformal** against your three exported backend checkpoints: Post-Placement, Post-CTS, and Post-Route using the provided (.do) file.

- **Analysis:** In your report, take a screenshot of the "Primitives Table" and "Compared Points" for the Post-Placement and Post-CTS runs. Explain why the primitive count (e.g., the number of Inverters or Buffers) changed between these two stages, even though the designs remained mathematically Equivalent.

Task 4: Physical Verification (LVS Signoff)

Launch the **Cadence PVS** LVS engine from the terminal using the provided `pegasus_ctrl.ctl` file to compare your final exported GDSII against the routed netlist.

- **Analysis:** Verify that your `lvs.rep` report shows a "**Matches**" status with exactly 0 incorrect nets and 0 incorrect instances. Review the `erc.sum` and `lvs.rep.shorts` files to confirm the absence of electrical rules violations and short circuits.

Task 4: Signoff STA (Tempus)

Launch **Cadence Tempus** and execute the script one by one using files generated by Innovus.

- **Timing Report:** Generate a final timing summary. Your design must achieve timing closure with zero slack violations for both Setup and Hold across all analyzed views.

Submission Requirements

Technical Analysis Report

- **Command Log:** Concise explanation of the purpose and output for each major command.
- **Metric Summary:** Final values for WNS/TNS, Clock Skew, Total Power, and LVS Match Status.



Visual Documentation

- **Layout Screenshots:** Capture the design state after each milestone:
 - Floorplan (Rings & Stripes)
 - Placement
 - CTS (Clock Tree Highlighted)
 - Routing
 - Final GDS (Fillers & Decaps included)

Reports & Design Files

- All required files generated by Innovus, PVS (LVS Reports) & Tempus.

7. Resources

- https://youtube.com/playlist?list=PLZU5hLL_713zf_i38C7uLu5pUz5wTjKul&si=2iMzUKy4YTXFFbG-
(Watch from Video #7 to 15)